# OperationNextNest:
# A Polyglot Adventure with Python + SAS + R

SAS On-the-Job Activity

## Purpose

This SAS On-the-Job activity demonstrates how modern analysts can move seamlessly between multiple programming languages within a single analytics environment. Using **SAS Viya Workbench for Learners**, participants will explore how Python, SAS, and R can work together to solve a complex real-world problem.

Your mission begins at **Home Finders Unlimited, LLC**, a subsidiary of iLink Mortgage, Inc. On your very first day, you are assigned to support a high-profile internal project: helping the company's CEO, Lincoln, identify the ideal neighborhood for his growing family.

To tackle this challenge, an experienced analyst has prepared three interconnected notebooks—one in **Python**, one in **SAS**, and one in **R**—each responsible for a different stage of the analytics workflow. As you work through the project, you will see how these tools complement one another while contributing to a shared objective.

At Home Finders Unlimited, the philosophy is simple: **use the right tool for the job—or more simply, use the tool you know.**

Because the project is both high-profile and confidential, it has been given a codename: **OperationNextNest.**

## Software

This SAS On-the-Job activity uses **SAS Viya Workbench for Learners**, an integrated development environment that supports multiple programming languages within a unified analytics platform.

Throughout the activity, you will work across two primary IDE experiences:

- **Visual Studio Code**
- **JupyterLab**

Within these environments, you will execute code using three separate kernels:

- **Python** – for data engineering and geographic data preparation
- **SAS** – to leverage PROC OPTMODEL so that we can better compare neighborhoods
- **R** – for data visualization and interactive geographic exploration

While this training is delivered using **SAS Viya Workbench**, the core code and concepts could be adapted to other environments that support Python, SAS, and R.

## Industry Alignment

This SAS On-the-Job activity aligns with **real estate analytics, location intelligence, and urban data analysis**.

Using publicly available data sources—including U.S. Census block group data, geographic boundaries, and community amenities—you will help identify neighborhoods that best meet the needs of a growing family. Along the way, you will explore techniques used by modern analysts to evaluate communities through data, such as:

- Analyzing demographic and housing characteristics
- Measuring commute distance and travel time
- Identifying neighborhood amenities such as parks and grocery stores
- Ranking neighborhoods using multiple criteria

By the end of the project, you will have contributed to a data-driven recommendation that helps guide an important family decision: **where to find the next nest.**

Copyright © 2026, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

# Table of Contents

*Purpose* 0

*Software* 0

*Industry Alignment* 1

**Mission Briefing: OperationNextNest** **4**

*Role: Data Analyst at Home Finders Unlimited, LLC* 4

*More Details on your Job* 4

*Your Assignment* 4

*Mission Setup: Preparing Your SAS Viya Workbench Environment* 5

**Part 1 | Data Engineering in Python** **10**

*Mission Background* 10

*Primary tool: Python* 10

*Objective* 10

*Notebook* 10

*What You'll Produce* 10

*Mission 1: Starting Now* 11
- Step 1 | Load Our Python Toolkit 11
- Step 2 | Define the Study Area 12
- Step 3 | Selecting Useful Census Variables 13
- Step 4 | Make the Data Human-Friendly 17
- Step 5 | Add Shapes: Block Group Boundaries 19
- Step 6 | Join Attributes to Shapes 21
- Step 7 | Quick Peek at the Map 23
- Step 8 | How Far Is Each Neighborhood from SAS? 27
- Step 9 | Neighborhood Amenities from OpenStreetMap 30
- Step 10 | Export for Analysis in SAS 33
- Part 1 Complete | Data Engineering in Python 36

**Part 2 | Neighborhood Scoring in SAS** **37**

*Mission Background* 37

*Primary tool: SAS* 37

*Objective* 38

*Notebook* 38

*What You'll Produce* 38

*Mission 2: Starting Now* 38
- Step 1 | Load the Dataset 39
- Step 2 | Prepare Comparison Variables 39
- Step 3 | Filter Candidates and Build Comparable Scores 42
- Step 4 | Run Scenario-Based Neighborhood Comparisons with PROC OPTMODEL 45
- Step 5 | Try Different "What Matters Most?" Scenarios 49

*Part 2 Complete | Neighborhood Scoring in SAS* 51

Copyright © 2026, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

Key Takeaways 52
Where We Are in the Workflow 52
Preparing for Geographic Exploration 52
Next Mission | Part 3 – Neighborhood Maps in R 53

**Part 3 | Neighborhood Maps in R 54**

*Mission Background 54*
From Tables to Maps 54
What We Gain from Mapping 54

*Primary tool: R 55*

*Objective 55*

*Notebook 55*

*What you'll produce 55*

*Mission 3: Let's Finish Strong! 56*
Step 1 | Environment Setup and Package Loading 56
Step 2 | Prepare the Data and Scenarios 57
Step 3 | Bring on the Geography 59
Step 4 | Exploring Neighborhood Scores with an Interactive Map 62
Step 5 | Identifying Consistent Neighborhoods Across Scenarios 64
Step 6 | Wrap-Up: What Did We Learn? 67
Mission Accomplished? 68

**Final Reflection 70**

*A Polyglot Approach to Analytics 70*

*What This Really Demonstrates 70*

*The Bigger Idea 71*

*Closing Thought 71*

**Appendix 72**

*Appendix A: Next Steps 72*

Copyright © 2026, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

# Mission Briefing: OperationNextNest

## Role: Data Analyst at Home Finders Unlimited, LLC

Welcome to your first day at **Home Finders Unlimited, LLC**, a subsidiary of **iLink Mortgage, Inc.**

The company specializes in using advanced analytics to help families make smarter housing decisions—from mortgage approvals to neighborhood recommendations. And today, you've been assigned to a particularly important internal project.

Your mission is to help the company's CEO, **Lincoln**, identify the best possible neighborhood for his rapidly expanding family.

The challenge? Finding the right neighborhood involves balancing many competing factors. Housing prices, commute times, neighborhood amenities, and demographic characteristics all play a role in determining whether a location is a good fit.

To tackle this challenge, the company launched a confidential analytics initiative with a fitting codename: **OperationNextNest.**

## More Details on your Job

As a new analyst, you are joining a team that believes strongly in **polyglot analytics**—using the right programming language for each stage of the analytical workflow.

Fortunately, a previous analyst has already begun the groundwork for this project. Before leaving for a well-earned vacation, they prepared three interconnected notebooks designed to guide the analysis:

- **Python** for data engineering and geographic preparation
- **SAS** to evaluate and ranking potential neighborhoods – using PROC OPTMODEL
- **R** for visualization and geographic storytelling

All our work takes place inside **SAS Viya Workbench for Learners**, where multiple languages can operate together within a single analytics environment.

Your task is not to start from scratch—but rather to **build upon the work that has already begun.**

## Your Assignment

As the newest analyst on the team, your goals are to:

- Access the notebooks prepared by your predecessor
- Understand how Python, SAS, and R work together in this workflow
- Modify and extend the analysis where appropriate
- Help identify neighborhoods that best match the needs of Lincoln and his family

Along the way, you will see how modern analytics teams combine multiple programming languages within a single platform to solve real-world problems.

Copyright © 2026, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

And don't worry—you won't be doing this alone. You'll have guidance from a very helpful onboarding buddy.

Which is me. Your humble narrator. And you can also use this Project Overview to guide you along the way:

**Operation Next Nest | Project Overview**

This project follows a simple question:

> **Where should Lincoln and his family look for their next home near SAS headquarters in Cary, North Carolina?**

To explore that question, we combine three analytics tools — each playing a different role in the workflow.

| Tool | Role | Purpose |
|---|---|---|
| Python | Data Engineering | Build the neighborhood dataset |
| SAS | Scenario Comparison | Evaluate neighborhoods under different priorities |
| R | Interactive Mapping | Visualize geographic patterns and robust candidates |

Together, these steps create a complete **data** → **decision** → **insight** pipeline for exploring real-world choices.

Let's begin **OperationNextNest!**

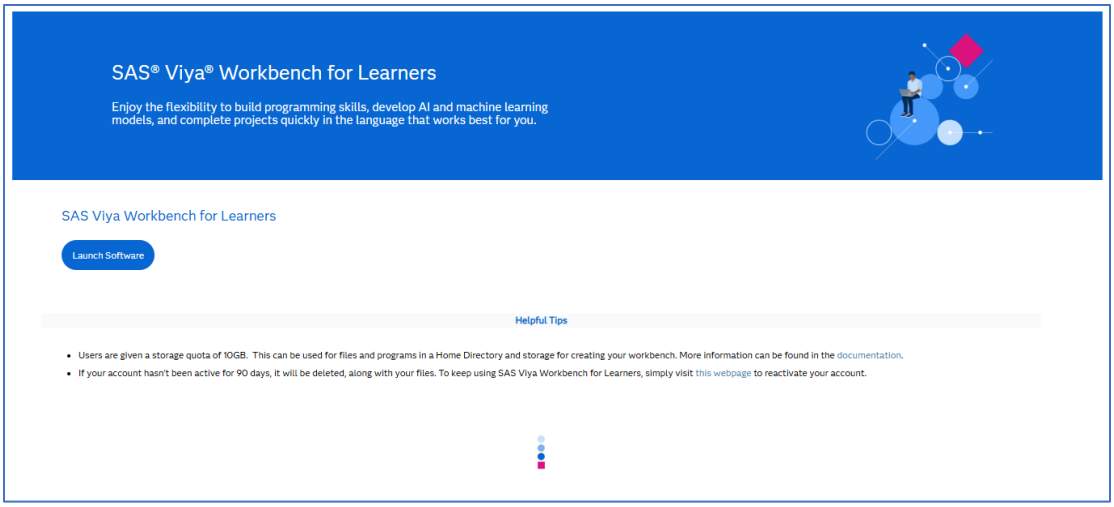
## Mission Setup: Preparing Your SAS Viya Workbench Environment

To begin this **polyglot adventure**, we first need to access **SAS Viya Workbench for Learners** and set up your development environment. This section walks you through the steps required to get everything running.

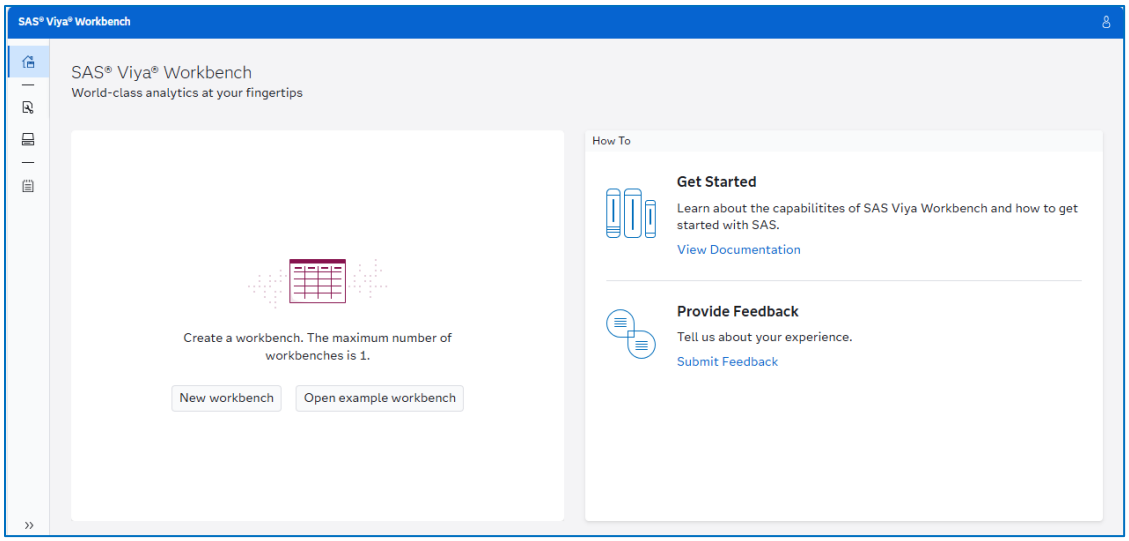
Follow the instructions carefully. Once the environment is ready, you will be just a few clicks away from running the programs that power **OperationNextNest** using **Python, SAS, and R**.

- To access SAS Viya Workbench for Learners, start on our product landing page, here: www.sas.com/workbenchforlearners
- Once you have a **SAS profile** and accept the terms of use, you should be taken to the SAS Viya Workbench for Learners product page, here: Course: SAS Viya Workbench for Learners. It looks like this:

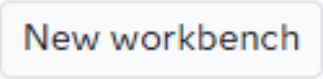
Copyright © 2026, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.




- Click the **Launch Software** button above to get started!
- If this is your first time in SAS Viya Workbench, you'll need to create a **New workbench**. And creation is just a few clicks away:

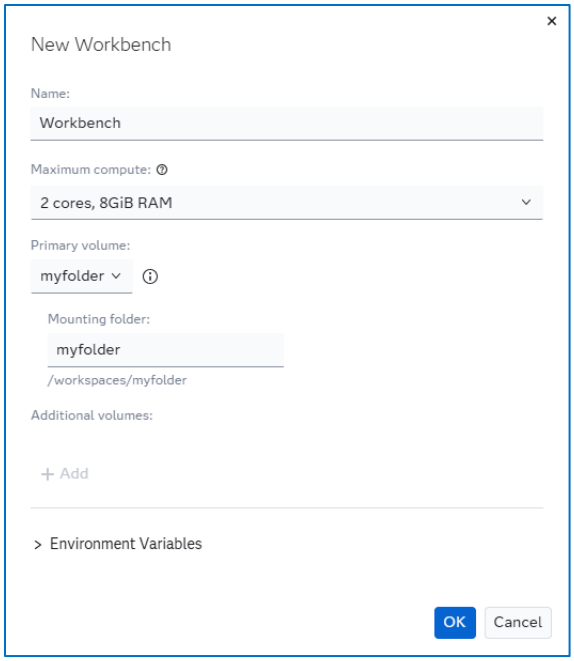



- Before creating the workbench, take a moment to explore the **WFL landing page**. For example, review the **Get Started** resources and the **Workbenches** and **Storage** options located in the left navigation panel. Becoming familiar with this interface will help you navigate the environment more confidently. And when you're satiated, click that **New workbench** button:

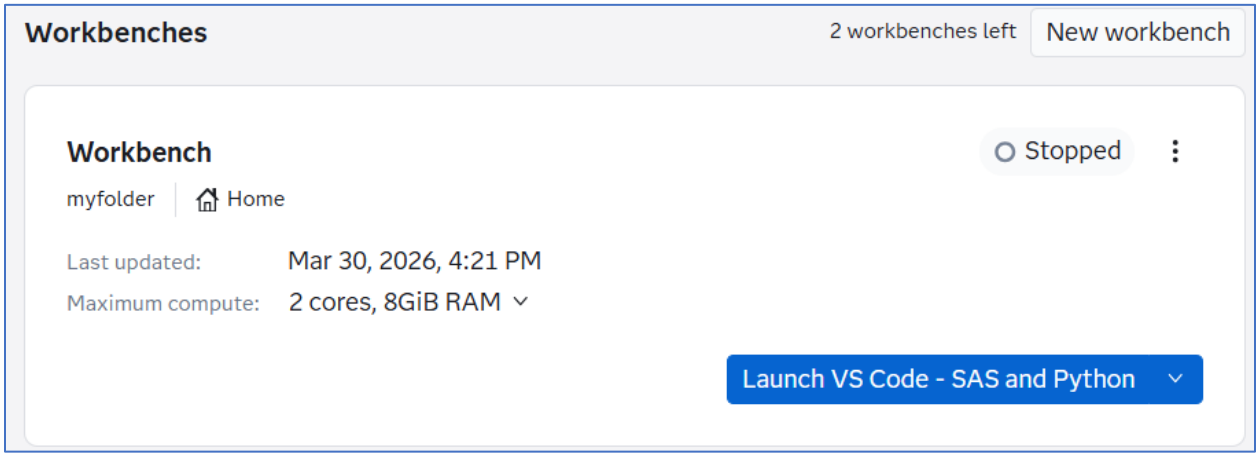



- The following appears:

Copyright © 2026, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.



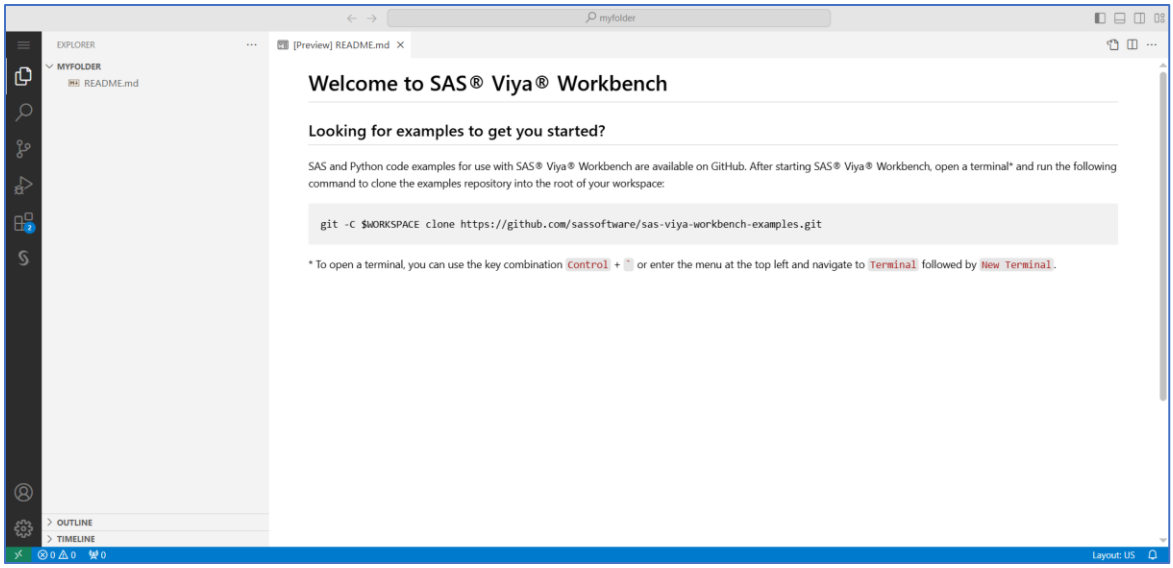

- For **Name**, let's just keep **Workbench** to simplify. And, in fact, just keep the other settings at their default and then click **OK**. Success looks like this:




- Next, in our new workbench, click the **Launch VS Code - SAS and Python** button, here:

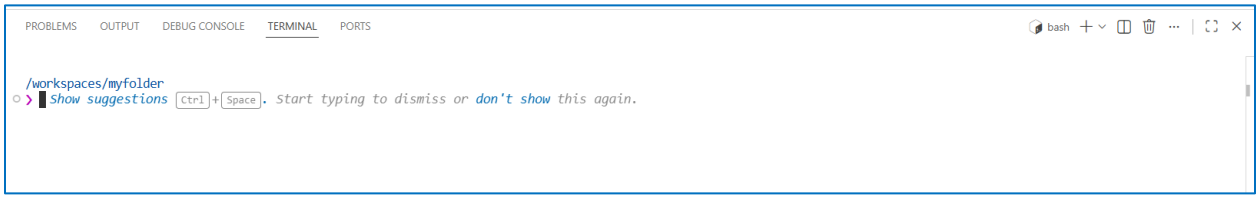
- Your workbench container loads! And welcome to **Visual Studio**!

Copyright © 2026, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.




- If this is your first time in the environment, explore the Visual Studio environment for a bit. Because we're all about exploring and learning here.
- Once you're more comfortable with Visual Studio, the last step in this section is to access the data for this SAS On-the-Job activity. You'll want to access the **Terminal** – which is likely hidden by default. No worries – it can be activated via the **Toggle Panel** if it's not already up. And that button is in the upper-righthand corner, here:

- And a happy terminal will look like the following:




- You're almost there! Simply type the following line of code into the Terminal:

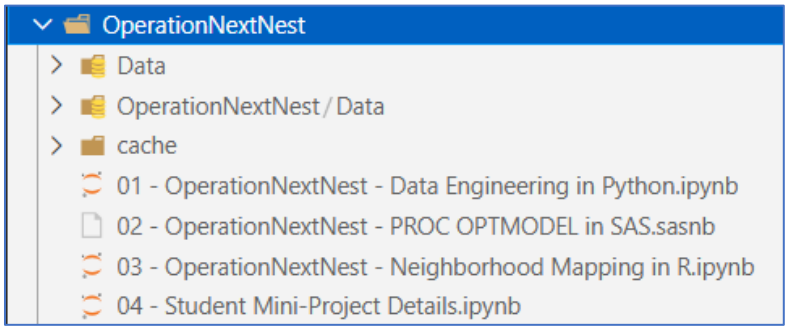
  git clone https://github.com/lincolngroves/OperationNextNest.git

- Then hit **Enter**. It's that easy to clone a GitHub repo and success looks like this:

Copyright © 2026, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

/workspaces/myfolder
❯ git clone https://github.com/lincolngroves/OperationNextNest.git
Cloning into 'OperationNextNest'...
remote: Enumerating objects: 123, done.
remote: Counting objects: 100% (20/20), done.
remote: Compressing objects: 100% (13/13), done.
remote: Total 123 (delta 10), reused 16 (delta 7), pack-reused 103 (from 1)
Receiving objects: 100% (123/123), 54.00 MiB | 46.86 MiB/s, done.
Resolving deltas: 100% (55/55), done.

/workspaces/myfolder 2s
❯
```

- One last important item for this section. Let's find the notebooks and data left behind by our predecessor! Access the **Explorer** button on the left pane and then expand the **OperationNextNest** folder to find the notebooks we'll use in this analysis. Like so:




- The three notebooks we will use in our analysis are:
  - 01 - OperationNextNest - Data Engineering in Python.ipynb
  - 02 - OperationNextNest - PROC OPTMODEL in SAS.sasnb
  - 03 - OperationNextNest - Neighborhood Mapping in R.ipynb
  - 04 - Student Mini-Project Details.ipynb.
- We will work through these notebooks in sequence…

Copyright © 2026, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

# Part 1 | Data Engineering in Python

## Mission Background

Welcome to the first stage of **OperationNextNest**.

In this mission, you will use **Python** to assemble the geographic and demographic data needed to evaluate neighborhoods across the Research Triangle region of North Carolina. This step focuses on **data engineering** — collecting, cleaning, and enriching multiple data sources so they can later be used in decision modeling.

Specifically, you will:

- Retrieve **Census block group data** for Wake and Durham counties
- Attach **geographic boundaries** for each neighborhood
- Estimate **commute distances** to the SAS campus in Cary, NC
- Identify nearby **amenities** such as parks, grocery stores, and bike trails using OpenStreetMap

By the end of this notebook, you will have created a clean, analysis-ready dataset describing neighborhoods across the region.

## Primary tool: Python

Python is particularly well suited for this stage of the project because of its rich ecosystem of libraries for working with **open data and geographic information**.

Packages such as pandas, geopandas, osmnx, and geopy allow us to integrate demographic data, geographic boundaries, and spatial analysis into a single workflow.

## Objective

Prepare the **geographic and demographic dataset** used to evaluate potential neighborhoods for Lincoln and his family.

This dataset will serve as the **foundation for the rest of the project**, enabling later modeling and visualization steps.

## Notebook

*01 – OperationNextNest – Data Engineering in Python.ipynb*

This notebook performs the full data engineering workflow required for the project.

## What You'll Produce

By the end of this mission, you will have created:

- A cleaned **Census block group dataset** for Wake and Durham counties
- Neighborhood **amenity indicators** (parks, grocery stores, bike trails)
- Estimated **commute distances and travel times** to the SAS campus
- A fully **analysis-ready dataset** for modeling in SAS

Copyright © 2026, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

## Mission 1: Starting Now

With the mission defined, it's time to open the notebook and begin assembling the data needed for OperationNextNest. Let's go!

### *Step 1 | Load Our Python Toolkit*

First, we install and import the Python packages that power **OperationNextNest**.

If you are running this notebook in a new **SAS Viya Workbench for Learners** environment, you may need to install these packages the first time the notebook runs. After the initial installation, the package installation line can usually be commented out.

Key packages used in this project include:

- **censusdata** → retrieve American Community Survey data from the Census API
- **pandas / numpy** → general data wrangling and numeric operations
- **geopandas** → spatial data structures and geographic analysis
- **geopy** → distance calculations between geographic points
- **osmnx** → retrieve neighborhood amenities from OpenStreetMap
- **matplotlib** → quick exploratory maps and visual checks

*The code:*

```
# If you are using SAS Viya Workbench for Learners and this is your first run
# in a new environment, you may need to install these packages once.
# After the first successful run, you can comment out this line.

!pip install censusdata geopandas matplotlib geopy osmnx --quiet

# Core packages for data access, wrangling, and mapping
import censusdata
import pandas as pd
import numpy as np
import geopandas as gpd

# Distance calculations and neighborhood amenities
from geopy.distance import geodesic
import osmnx as ox

# Visualization
import matplotlib.pyplot as plt

# Display settings
pd.set_option("display.max_columns", 50)
```

Once these packages are installed and imported, the notebook is ready to begin retrieving and engineering the data needed for OperationNextNest.

Copyright © 2026, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

### *Step 2 | Define the Study Area*

Before retrieving data, we first need to define **where the analysis will take place**.

Lincoln and his family would like to remain in the **Research Triangle region of North Carolina**, ideally within commuting distance of the **SAS campus in Cary, NC**. The Triangle region is anchored by the cities of Raleigh, Durham, and Chapel Hill and is known for its strong economy, universities, and technology sector.

And why focus on SAS campus? Well, Lincoln has a created a fake empire through all his side hustles, which includes **Home Finders Unlimited, LLC**, a subsidiary of iLink Mortgage, Inc. Despite these activities, his primary job is still at the SAS Institute… so we'll focus our search around there.

To keep the analysis focused, we will limit our study area to two counties:

- **Wake County (FIPS: 183)**
- **Durham County (FIPS: 063)**

These identifiers are known as **FIPS codes** (Federal Information Processing Standards). FIPS codes provide a standardized way to reference geographic areas in many U.S. datasets, including the **American Community Survey (ACS)** used in this project.

#### *Why Use the American Community Survey?*

The **American Community Survey (ACS)** is an ongoing survey conducted by the U.S. Census Bureau that provides detailed demographic, housing, and economic information about communities across the United States.

In this project we use the **2021 ACS 5-year dataset**, which aggregates survey responses collected between **2017 and 2021**.

Although ACS data is released every year, the **5-year estimates are the only ACS dataset available at the Census block group level**, making them particularly useful for neighborhood-level analysis.

#### *Why Census Block Groups?*

Our analysis focuses on **Census block groups**, which are small geographic areas that typically contain **600 to 3,000 residents**.

Block groups are detailed enough to capture meaningful differences between neighborhoods while still providing reliable estimates from the ACS.

By combining **block group demographics** with **geographic boundaries**, we can analyze neighborhoods across the region in a consistent way.

#### *What These Geographic Boundaries Enable*

Once the study area is defined, these geographic boundaries will allow us to:

- Retrieve **Census demographic data** for each block group

Copyright © 2026, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

- Map **neighborhood boundaries** later in the project
- Calculate **commute distances** from each neighborhood to the SAS campus

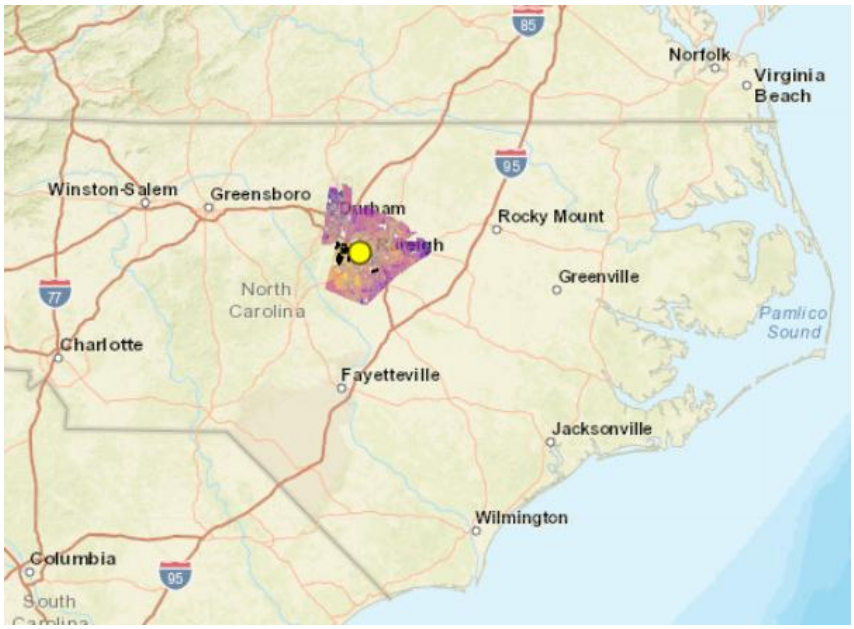
With the study area defined, we can now begin pulling Census data for the neighborhoods that fall inside these counties.

The first step is to define the state and county identifiers used in our Census queries.

*The Code:*

```
# ------------------------------------------------------------------
# Study area: North Carolina block groups in Wake and Durham Counties
# FIPS = Federal Information Processing Standards geographic codes
# ------------------------------------------------------------------
STATE_FIPS = "37"        # North Carolina
WAKE_FIPS = "183"        # Wake County
DURHAM_FIPS = "063"      # Durham County

COUNTY_FIPS = [WAKE_FIPS, DURHAM_FIPS]
```

*A Reference Point:*




*The general search area in North Carolina*

## *Step 3 | Selecting Useful Census Variables*

With the study area defined, the next step is to determine **which Census variables will help us evaluate potential neighborhoods**.

The **American Community Survey (ACS)** contains thousands of variables covering topics such as housing, demographics, education, employment, and transportation. Rather than retrieving

Copyright © 2026, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

everything, analysts typically select a focused set of indicators that reflect the questions they want to answer.

For OperationNextNest, we are particularly interested in variables that capture:

- Housing affordability
- Economic conditions
- Family characteristics
- Transportation patterns

These indicators help describe neighborhoods from the perspective of a family deciding where to live.

### *Key Variables Used in This Project*

The table below highlights several of the most important variables we retrieve from the ACS dataset.

| Category | Census Variable | Why It Matters |
|---|---|---|
| **Housing Affordability** | Median Housing Value | Indicates typical home prices in the neighborhood |
| **Housing Costs** | Median Gross Rent | Useful proxy for overall housing affordability |
| **Housing Costs** | Monthly Owner Costs (with Mortgage) | Helps approximate the cost of homeownership |
| **Economic Conditions** | Median Household Income | Indicator of local economic strength |
| **Family Characteristics** | Households with Children Under 6 | Proxy for kid-friendly neighborhoods |
| **Family Characteristics** | Married-Couple Households | Helps describe neighborhood composition |
| **Transportation** | Mean Travel Time to Work | Baseline commute patterns in the area |

### *Why Use Block Group Data?*

These variables are retrieved at the **Census block group level**, which provides the most detailed geographic resolution available in the ACS 5-year dataset.

Each block group is uniquely identified by a **12-digit GEOID**, which combines:

- State (2 digits)

Copyright © 2026, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

- County (3 digits)
- Census Tract (6 digits)
- Block Group (1 digit)

This identifier allows us to later **merge demographic data with geographic boundaries** and compute neighborhood-level features such as commute distance and amenities.

### *Building the Block Group Dataset*

To retrieve the data, the notebook defines a helper function that queries the Census API and constructs the GEOID for each block group.

The function is then used to retrieve data separately for **Wake County** and **Durham County**, which are combined into a single dataset representing all block groups in our study area.

**What This Step Produces**

At the end of this step, we now have a dataset where:

- Each **row represents a Census block group**
- Each **column represents a demographic or housing feature**
- Every row is uniquely identified using **GEOID**

This dataset forms the **demographic backbone** of the OperationNextNest project.

### *The Code:*

```
# ------------------------------------------------------------------
# ACS = American Community Survey
# We will use 2021 ACS 5-year estimates at the block group level.
# The 5-year dataset is used because block-group data are available
# in the ACS 5-year release.
#
# Variable definitions:
# https://api.census.gov/data/2021/acs/acs5/variables.html
# ------------------------------------------------------------------
ACS_YEAR = "2021"

ACS_VAR_LABELS = {
    "B01001_003E": "Male under 5 years",
    "B01001_027E": "Female under 5 years",
    "B01003_001E": "Total population",
    "B11003_003E": "Family households with children under 6 years",
    "B11001_002E": "Total family households",
    "B11003_002E": "Households with children under 18",
    "B11004_002E": "Married-couple family households",
    "B11004_004E": "Married-couple family households with children under 6 years",
    "B19013_001E": "Median household income",
    "B25064_001E": "Median gross rent",
    "B25088_002E": "Median monthly owner costs (with mortgage)",
    "B25077_001E": "Median housing value",
    "B25034_001E": "Median year structure built",
    "B25002_003E": "Vacant housing units",
    "B14003_003E": "Nursery school enrollment",
    "B14003_004E": "Kindergarten enrollment",
    "B15003_017E": "Some college",
    "B15003_022E": "Bachelor's degree or higher",
    "B08013_001E": "Mean travel time to work (minutes)",
    "B25044_003E": "Households with no vehicle"
}

ACS_VARS = list(ACS_VAR_LABELS.keys())
```

Copyright © 2026, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

```python
def download_block_groups(county_fips: str) -> pd.DataFrame:
    """
    Download 2021 ACS 5-year data for all Census block groups in a county.

    Returns
    -------
    pandas.DataFrame
        A DataFrame containing ACS variables plus a 12-digit GEOID:
        state (2) + county (3) + tract (6) + block group (1)
    """
    df = censusdata.download(
        "acs5",
        int(ACS_YEAR),
        censusdata.censusgeo([
            ("state", STATE_FIPS),
            ("county", county_fips),
            ("block group", "*")
        ]),
        ACS_VARS
    ).reset_index()

    # Build a standard 12-digit GEOID for joining to shapefiles and maps
    df["GEOID"] = df["index"].apply(
        lambda x: (
            x.geo[0][1].zfill(2) +  # state
            x.geo[1][1].zfill(3) +  # county
            x.geo[2][1].zfill(6) +  # tract
            x.geo[3][1].zfill(1)    # block group
        )
    )

    return df.drop(columns=["index"])
```

```python
# Download block-group data for each county in the study area
wake_data = download_block_groups(WAKE_FIPS)
durham_data = download_block_groups(DURHAM_FIPS)

# Combine the county-level files into one study-area dataset
census_blockgroup_data = pd.concat([wake_data, durham_data], ignore_index=True)

# Preview the first few rows
census_blockgroup_data.head()
```

*Partial Output:*

| | B01001_003E | B01001_027E | B01003_001E | B11003_003E | B11001_002E | B11003_002E | B11004_002E | B11004_004E | B19013_001E | B25064_001E |
|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 31 | 25 | 1203 | 45 | 145 | 134 | 134 | 22 | 97917 | 1238 |
| 1 | 9 | 22 | 2874 | 18 | 285 | 265 | 265 | 8 | 101058 | 1788 |
| 2 | 0 | 0 | 1110 | 0 | 134 | 107 | 107 | 0 | 109196 | 1778 |
| 3 | 0 | 17 | 1723 | 73 | 250 | 230 | 230 | 18 | 67536 | 996 |
| 4 | 0 | 0 | 605 | 0 | 35 | 31 | 31 | 0 | 72083 | 1259 |

*Another Check:*

```python
# One more check
print(f"Number of block groups in study area: {len(census_blockgroup_data):,}")
print(f"Number of columns: {census_blockgroup_data.shape[1]}")
```

✓ 0.0s

```
Number of block groups in study area: 835
Number of columns: 21
```

Copyright © 2026, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

Next, we will transform the raw Census variables into more interpretable measures that help characterize family-friendly neighborhoods.

### *Step 4 | Make the Data Human-Friendly*

The Census variables retrieved in the previous step use official **ACS variable codes**, such as:

B19013_001E

These codes are necessary when querying the Census API, but they are not very useful when performing analysis or interpreting results.

To make the dataset easier to work with, we now transform the data into a more **human-friendly format**.

Specifically, we will:

- Assign **clear, descriptive column names** to the ACS variables
- Create several **percentage-based indicators** that make neighborhoods easier to compare

These new indicators include:

- Percent of children under 5
- Percent of households with children under 6
- Percent of married-couple households
- Percent of married-couple households with young children

Many Census variables are reported as **counts**, but counts alone can be misleading when comparing neighborhoods of different sizes. By converting these counts into **percentages**, we can more easily compare demographic patterns across block groups.

These derived measures will later help us evaluate which neighborhoods may be more **family-friendly**, an important consideration for Lincoln and his growing family.

#### *Transforming Census Variables*

In the notebook, the code performs two main tasks:

1. **Rename Census variables** to more descriptive column names
2. **Compute percentage-based indicators** using the population and household counts retrieved earlier

#### *What This Step Produces*

At the end of this step, the dataset becomes much easier to interpret. Instead of raw Census codes and counts, we now have variables such as:

- Percent of children under 5
- Percent of households with young children
- Percent of married households

Copyright © 2026, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

These indicators begin to paint a clearer picture of the **demographic character of each neighborhood**.

*The Code:*

```
# Rename ACS columns to cleaner, analysis-friendly names
rename_map = {
    "B01001_003E": "male_under_5",
    "B01001_027E": "female_under_5",
    "B01003_001E": "total_population",
    "B11003_003E": "hh_children_under_6",
    "B11001_002E": "total_family_households",
    "B11003_002E": "hh_children_under_18",
    "B11004_002E": "married_couple_households",
    "B11004_004E": "married_couple_children_under_6",
    "B19013_001E": "median_household_income",
    "B25064_001E": "median_gross_rent",
    "B25088_002E": "monthly_owner_costs_mortgage",
    "B25077_001E": "median_housing_value",
    "B25034_001E": "median_year_structure_built",
    "B25002_003E": "vacant_housing_units",
    "B14003_003E": "nursery_school_enrollment",
    "B14003_004E": "kindergarten_enrollment",
    "B15003_017E": "some_college",
    "B15003_022E": "bachelors_degree_or_higher",
    "B08013_001E": "mean_travel_time_to_work",
    "B25044_003E": "households_no_vehicle"
}

census_blockgroup_data = census_blockgroup_data.rename(columns=rename_map)

# Derived counts and percentages
census_blockgroup_data["children_under_5"] = (
    census_blockgroup_data["male_under_5"] +
    census_blockgroup_data["female_under_5"]
)

# Guard against divide-by-zero when creating percentage-based variables
census_blockgroup_data["prc_children_under_5"] = np.where(
    census_blockgroup_data["total_population"] > 0,
    census_blockgroup_data["children_under_5"] / census_blockgroup_data["total_population"] * 100,
    np.nan
)
```

```
census_blockgroup_data["prc_hh_children_under_6"] = np.where(
    census_blockgroup_data["total_family_households"] > 0,
    census_blockgroup_data["hh_children_under_6"] /
    census_blockgroup_data["total_family_households"] * 100,
    np.nan
)

census_blockgroup_data["prc_married_couple_households"] = np.where(
    census_blockgroup_data["total_family_households"] > 0,
    census_blockgroup_data["married_couple_households"] /
    census_blockgroup_data["total_family_households"] * 100,
    np.nan
)

census_blockgroup_data["prc_married_couple_children_under_6"] = np.where(
    census_blockgroup_data["total_family_households"] > 0,
    census_blockgroup_data["married_couple_children_under_6"] /
    census_blockgroup_data["total_family_households"] * 100,
    np.nan
)

census_blockgroup_data.head()
```

Copyright © 2026, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

*Partial Output:*

| | male_under_5 | female_under_5 | total_population | hh_children_under_6 | total_family_households | hh_children_under_18 |
|---|---|---|---|---|---|---|
| 0 | 31 | 25 | 1203 | 45 | 145 | 134 |
| 1 | 9 | 22 | 2874 | 18 | 285 | 265 |
| 2 | 0 | 0 | 1110 | 0 | 134 | 107 |
| 3 | 0 | 17 | 1723 | 73 | 250 | 230 |
| 4 | 0 | 0 | 605 | 0 | 35 | 31 |

With the demographic variables prepared, we can now add geographic boundaries that allow us to map and analyze neighborhoods spatially.

### *Step 5 | Add Shapes: Block Group Boundaries*

So far, our dataset contains **demographic and housing statistics** for neighborhoods across Wake and Durham counties.

However, these values exist only as **tabular data**. To create maps and perform spatial analysis, we also need to know **where these neighborhoods are located geographically**.

To accomplish this, we will attach **geographic boundary data** to our Census dataset.

#### *Census Geography and TIGER/Line Files*

The U.S. Census Bureau publishes geographic boundary files known as **TIGER/Line shapefiles**. These files define the shapes of many different geographic units used by the Census, including:

- States
- Counties
- Census tracts
- Census block groups

Each block group boundary is represented as a **polygon**, which describes the physical shape of the neighborhood on a map. Block groups typically contain **600 to 3,000 residents**, making them a useful geographic unit for neighborhood-level analysis.

#### *From DataFrame to GeoDataFrame*

Once we attach the geographic shapes to our dataset, the structure changes slightly.

Instead of working with a standard **pandas DataFrame**, we will now work with a **GeoDataFrame**.

A GeoDataFrame is similar to a normal table, but it includes a special column called **geometry**, which stores the spatial shape associated with each row.

After this step:

- Each row still represents a **Census block group**
- The dataset now includes a **polygon describing the neighborhood's geographic boundary**

Copyright © 2026, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

This allows us to perform spatial operations such as mapping, distance calculations, and spatial joins.

### *Coordinate Reference System (CRS)*

Before performing spatial analysis, we also ensure that the geographic data uses the **WGS84 coordinate reference system (EPSG:4326)**.

This system represents locations using **latitude and longitude**, and it is the standard coordinate system used by most web mapping tools, including **Leaflet**, which we will use later in the R portion of the project.

### *What This Step Does*

In the notebook, the code performs two key tasks:

1. Load the **TIGER/Line block group shapefile for North Carolina**
2. Filter the shapefile to include only **Wake and Durham counties** using their FIPS codes

### *What This Step Produces*

At the end of this step, we now have a **GeoDataFrame** containing:

- Census demographic variables
- Housing indicators
- A **geometry column** describing the boundaries of each block group

This geographic structure allows us to begin performing **spatial analysis and mapping** in the next steps of the project.

### *The Code:*

```
# ------------------------------------------------------------
# Load the TIGER/Line block group shapefile for North Carolina
# ------------------------------------------------------------

# Path to the NC block group shapefile
shapefile_path = "./tl_2023_37_bg.shp"

# Read the shapefile using GeoPandas
gdf = gpd.read_file(shapefile_path)

# Confirm the coordinate reference system (CRS)
print("CRS:", gdf.crs)

# Filter to keep only Wake and Durham counties
gdf = gdf[gdf["COUNTYFP"].isin(COUNTY_FIPS)].copy()

# Create a 12-digit GEOID to match the ACS dataset
gdf["GEOID"] = gdf["GEOID"].astype(str)

# Preview the geographic dataset
gdf.head()
```

Copyright © 2026, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

*Partial Output:*

| | STATEFP | COUNTYFP | TRACTCE | BLKGRPCE | GEOID | GEOIDFQ | NAMELSAD | MTFCC | FUNCSTAT | ALAND |
|---|---|---|---|---|---|---|---|---|---|---|
| 92 | 37 | 063 | 002019 | 1 | 370630020191 | 1500000US370630020191 | Block Group 1 | G5030 | S | 4885136 |
| 93 | 37 | 063 | 002020 | 1 | 370630020201 | 1500000US370630020201 | Block Group 1 | G5030 | S | 7053910 |
| 94 | 37 | 063 | 001806 | 1 | 370630018061 | 1500000US370630018061 | Block Group 1 | G5030 | S | 3377611 |
| 95 | 37 | 063 | 001808 | 2 | 370630018082 | 1500000US370630018082 | Block Group 2 | G5030 | S | 4429008 |
| 96 | 37 | 063 | 002024 | 1 | 370630020241 | 1500000US370630020241 | Block Group 1 | G5030 | S | 586893 |

Now that the geographic boundaries are attached, we can combine the Census attributes and the block group shapes into a single spatial dataset.

### *Step 6 | Join Attributes to Shapes*

Up to this point in the project, we have created two separate datasets:

- **census_blockgroup_data** — a table containing demographic and housing attributes for each neighborhood
- **gdf** — a spatial dataset containing the geographic shapes (polygons) of Census block groups

Both datasets describe the same geographic units, but they currently exist **independently**.

To perform spatial analysis and mapping, we need to combine these two pieces of information into a single dataset.

#### *Joining the Datasets Using GEOID*

The key to merging these datasets is the **GEOID**.

As discussed earlier, the GEOID is a **12-digit Census identifier** that uniquely identifies each block group in the United States. Because both datasets contain this identifier, we can use it as a **join key** to merge the tables together.

This process is conceptually similar to joining two tables in SQL.

#### *Creating a GeoDataFrame with Both Attributes and Geometry*

After performing the merge, we create a new object called **merged_gdf**.

This object is a **GeoDataFrame**, which now contains:

- the **demographic attributes** retrieved from the American Community Survey
- the **polygon geometry** describing the shape of each block group

Each row in this dataset now represents a neighborhood that knows both:

- **where it is located geographically**, and
- **what it looks like demographically**

This combined structure allows us to perform spatial analysis and create maps in the next steps of the project.

Copyright © 2026, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

*Why This Join Matters*

Joining attributes to geometry is one of the most common operations in **geospatial analysis**.

Once attributes and shapes are combined, analysts can:

- visualize data on maps
- perform spatial calculations
- analyze geographic patterns across regions

This step effectively transforms our dataset from a collection of **demographic statistics** into a dataset describing **real neighborhoods on the map**.

*The Code:*

```
# Ensure GEOID is stored as a string in both datasets
gdf["GEOID"] = gdf["GEOID"].astype(str)
census_blockgroup_data["GEOID"] = census_blockgroup_data["GEOID"].astype(str)

# Join ACS attributes to block group geometries
merged_gdf = gdf.merge(
    census_blockgroup_data,
    on="GEOID",
    how="left"
)

# Confirm the result is still a GeoDataFrame
merged_gdf = gpd.GeoDataFrame(merged_gdf, geometry="geometry", crs=gdf.crs)

# Preview the merged dataset
merged_gdf.head()
```

*Partial Output:*

| | STATEFP | COUNTYFP | TRACTCE | BLKGRPCE | GEOID | GEOIDFQ | NAMELSAD | MTFCC | FUNCSTAT | ALAND |
|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 37 | 063 | 002019 | 1 | 370630020191 | 1500000US370630020191 | Block Group 1 | G5030 | S | 4885136 |
| 1 | 37 | 063 | 002020 | 1 | 370630020201 | 1500000US370630020201 | Block Group 1 | G5030 | S | 7053910 |
| 2 | 37 | 063 | 001806 | 1 | 370630018061 | 1500000US370630018061 | Block Group 1 | G5030 | S | 3377611 |
| 3 | 37 | 063 | 001808 | 2 | 370630018082 | 1500000US370630018082 | Block Group 2 | G5030 | S | 4429008 |
| 4 | 37 | 063 | 002024 | 1 | 370630020241 | 1500000US370630020241 | Block Group 1 | G5030 | S | 586893 |

Copyright © 2026, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

*Validation Check:*

```
# Tiny validation check:
print(f"Rows in shapefile: {len(gdf):,}")
print(f"Rows in merged file: {len(merged_gdf):,}")
print(f"Missing ACS matches: {merged_gdf['total_population'].isna().sum():,}")
```
✓ 0.0s

```
Rows in shapefile: 835
Rows in merged file: 835
Missing ACS matches: 0
```

Now that the demographic attributes and geographic shapes are combined, we can perform a quick visual check to confirm that the data behave as expected.

### *Step 7 | Quick Peek at the Map*

Before moving on to additional feature engineering, it is helpful to perform a quick **visual sanity check**. At this stage of the project, we have successfully combined:

- Census demographic attributes
- Block group geographic boundaries

Because these attributes and shapes now exist in the same GeoDataFrame, we can already begin creating simple maps. Visualizing the data early helps confirm that everything is behaving as expected.

#### *Why Perform a Visual Sanity Check?*

Quick exploratory maps help analysts verify that:

- The **block group boundaries** appear in the correct geographic locations
- The **demographic variables** vary across neighborhoods as expected
- No issues occurred during the **data merge or transformation steps**

In other words, these maps help ensure that our dataset is **structurally sound before moving forward**.

#### *Creating Choropleth Maps*

To explore the data, we will create several **choropleth maps**.

A choropleth map uses color shading to represent the value of a variable across geographic areas. Darker or lighter colors indicate higher or lower values depending on the color scale used.

The notebook creates quick visualizations for several variables, including:

- prc_hh_children_under_6
- median_housing_value
- median_year_structure_built
- median_household_income

Copyright © 2026, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

- monthly_owner_costs_mortgage

These variables capture different aspects of neighborhood characteristics, including **family composition, housing costs, and economic conditions**.

### *Adding a Geographic Reference Point*

To make the maps easier to interpret, we also mark the **SAS campus** on each map.

This provides a clear geographic reference point for understanding where neighborhoods are located relative to Lincoln's workplace.

### *What to Look For*

When examining exploratory maps like these, analysts typically look for patterns such as:

- Areas where values appear **unexpectedly uniform**
- Regions with **missing data**
- Variables that show **very little variation across neighborhoods**

If a variable lacks variation or contains many missing values, it may not be very useful for later analysis.

This step helps identify potential issues early, before the dataset is used in our neighborhood analysis.

### *The Code:*

```
# Variables to map: {column_name: display_label}
variables = {
    "prc_hh_children_under_6": "Percent of Households with Children under 6",
    "median_housing_value": "Median Housing Value",
    "median_year_structure_built": "Median Year Structure Built",
    "median_household_income": "Median Household Income",
    "monthly_owner_costs_mortgage": "Monthly Owner Costs (with mortgage)"
}

# Custom display ranges for selected variables
custom_scales = {
    "prc_hh_children_under_6": {
        "vmin": 0,
        "vmax": 100,
        "ticks": np.arange(0, 101, 25)
    },
    "median_housing_value": {
        "vmin": 0,
        "vmax": 1_500_000,
        "ticks": np.arange(0, 1_500_001, 250_000)
    },
    "median_year_structure_built": {
        "vmin": 1950,
        "vmax": 2025,
        "ticks": np.arange(1950, 2026, 25)
    },
    "median_household_income": {
        "vmin": 0,
        "vmax": 1_000_000,
        "ticks": np.arange(0, 1_000_001, 200_000)
    },
    "monthly_owner_costs_mortgage": {
        "vmin": 0,
        "vmax": 10_000,
        "ticks": np.arange(0, 10_001, 2_000)
    }
}
```

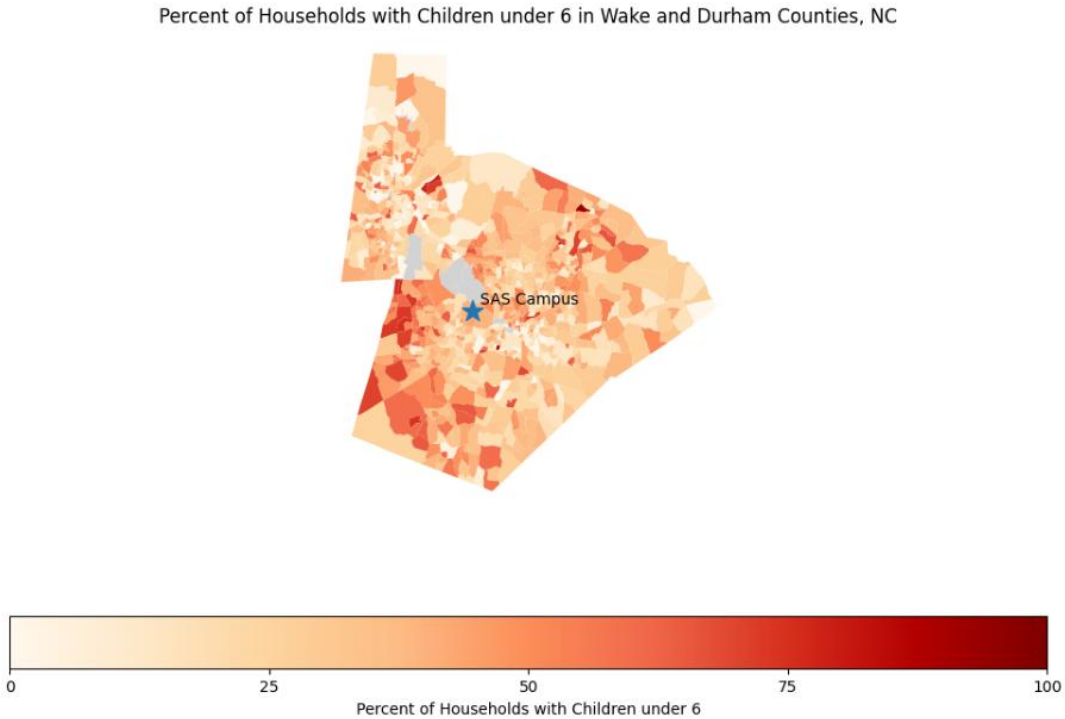
Copyright © 2026, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

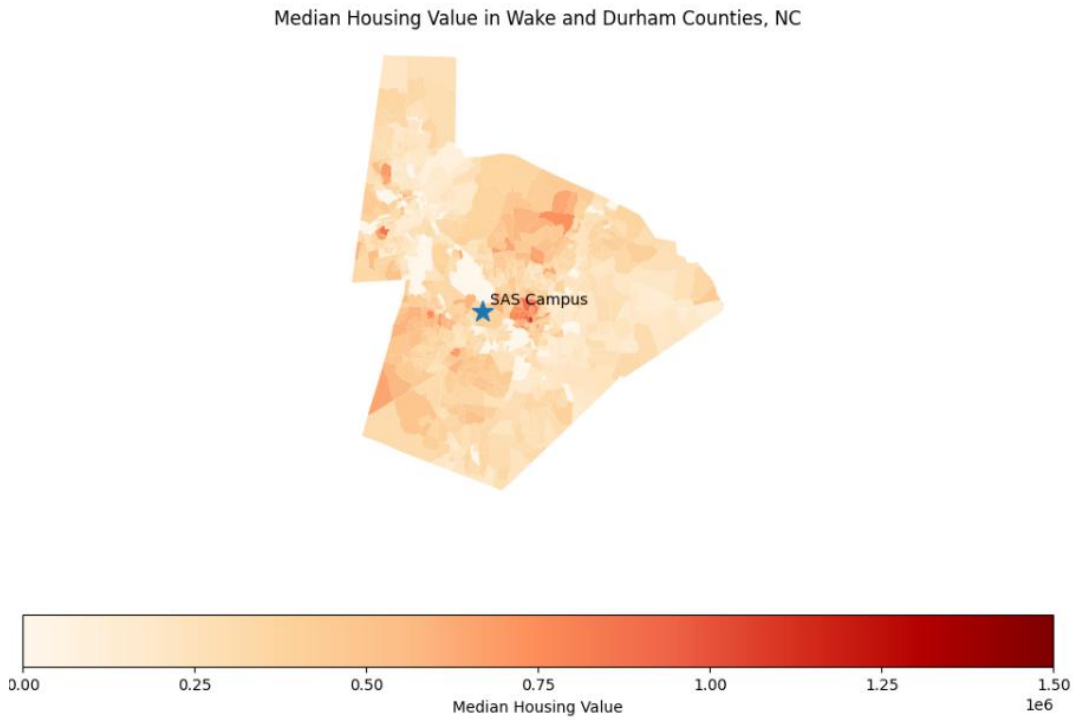
```python
# SAS campus location: longitude, latitude
sas_lon, sas_lat = -78.74939336654782, 35.81563890659871

for var, label in variables.items():
    scale = custom_scales.get(var, {})
    vmin = scale.get("vmin")
    vmax = scale.get("vmax")
    ticks = scale.get("ticks")

    fig, ax = plt.subplots(figsize=(12, 8))

    merged_gdf.plot(
        column=var,
        cmap="OrRd",
        legend=True,
        vmin=vmin,
        vmax=vmax,
        missing_kwds={"color": "lightgrey", "label": "Missing"},
        legend_kwds={
            "label": label,
            "orientation": "horizontal",
            "ticks": ticks
        },
        ax=ax
    )

    # Mark the SAS campus
    ax.scatter(sas_lon, sas_lat, marker="*", s=200)

    # Add a text label next to the star
    ax.annotate(
        "SAS Campus",
        xy=(sas_lon, sas_lat),
        xytext=(5, 5),
        textcoords="offset points",
        fontsize=10
    )
```

```python
# Clean up the map display
ax.set_title(f"{label} in Wake and Durham Counties, NC")
ax.set_axis_off()

plt.show()
```

Copyright © 2026, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

*Partial Output:*




With the demographic and geographic data validated, we can now begin calculating commute distances between each neighborhood and the SAS campus.

Copyright © 2026, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

### *Step 8 | How Far Is Each Neighborhood from SAS?*

Commute time is one of the most important factors when choosing where to live. Even if a neighborhood is affordable and family-friendly, a long commute can quickly make it impractical.

In this step, we estimate how far each neighborhood is from the **SAS campus in Cary, North Carolina**, which serves as Lincoln's workplace (and fake empire HQ).

To do this, we answer two questions:

- How far (in miles) is each block group's **centroid** from the SAS campus?
- If we assume an average travel speed of **35 miles per hour**, what is the estimated **travel time**?

#### *What Is a Centroid?*

A **centroid** represents the geographic center of a polygon.

In our case, each Census block group is represented by a polygon describing its geographic boundary. The centroid acts as a practical stand-in for the location of that neighborhood. Using the centroid allows us to approximate the distance between the **center of a neighborhood** and **SAS campus**.

#### *Straight-Line Distance vs. Driving Distance*

In this notebook, we estimate commute distance using **straight-line distance**, sometimes referred to as "as-the-crow-flies" distance.

This approach is simple and computationally efficient, but it is important to recognize that it is a **simplification**.

In real-world applications, analysts often estimate travel time using routing APIs such as:

- Google Maps Distance Matrix API
- OpenRouteService
- Mapbox Directions API

These tools estimate travel time using **actual road networks and traffic patterns**.

For this learning activity, however, the centroid-based method provides a transparent and reproducible way to estimate commute distance without requiring external API keys or additional setup.

#### *Estimating Travel Time*

Once the distance between each block group centroid and the SAS campus is calculated, we estimate travel time using a simple assumption:

Travel Time = Distance ÷ Speed

For this exercise, we assume an **average travel speed of 35 miles per hour**.

Copyright © 2026, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

This produces a rough estimate of how long it might take to travel from each neighborhood to the SAS campus.

*Why This Metric Matters*

The commute metric will become an important feature in the later stages of the project.

Students will be able to:

- Adjust **travel-related weights** in the neighborhood scoring model
- Apply **maximum commute constraints**
- Explore how different commute preferences affect the final neighborhood rankings

In this way, the commute variable helps connect the data engineering work in Python with the **decision modeling performed in SAS**.

*What This Step Produces*

At the end of this step, the dataset will now include:

- **Distance to Office (miles)**
- **Estimated Travel Time to Office (minutes)**

These variables describe how accessible each neighborhood is relative to Lincoln's workplace.

*The Code:*

```
# ------------------------------------------------------------
# Estimate commute distance and travel time to the SAS campus
# ------------------------------------------------------------

# SAS campus location in (latitude, longitude)
office_coords = (35.81563890659871, -78.74939336654782)

# Re-project to a projected CRS for more accurate centroid calculation
# EPSG:32119 = NAD83 / North Carolina
merged_gdf_projected = merged_gdf.to_crs(epsg=32119).copy()

# Calculate block group centroids in the projected CRS
merged_gdf_projected["centroid"] = merged_gdf_projected.geometry.centroid

# Create a GeoDataFrame of centroids and convert back to WGS84
centroids_gdf = gpd.GeoDataFrame(
    merged_gdf_projected[["GEOID", "centroid"]].copy(),
    geometry="centroid",
    crs=merged_gdf_projected.crs
).to_crs(epsg=4326)

# Extract centroid coordinates as (latitude, longitude)
centroids_gdf["centroid_lat"] = centroids_gdf.geometry.y
centroids_gdf["centroid_lon"] = centroids_gdf.geometry.x
centroids_gdf["centroid_lat_lon"] = list(
    zip(centroids_gdf["centroid_lat"], centroids_gdf["centroid_lon"])
)
```

Copyright © 2026, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

```
print(centroids_gdf[["centroid_lat", "centroid_lon"]].head())

# Calculate straight-line distance to the SAS campus in miles
centroids_gdf["distance_to_office_miles"] = centroids_gdf["centroid_lat_lon"].apply(
    lambda coords: geodesic(coords, office_coords).miles
)

# Estimate travel time in minutes using an average speed assumption
average_speed_mph = 35
centroids_gdf["travel_time_to_office_min"] = (
    centroids_gdf["distance_to_office_miles"] / average_speed_mph * 60
)

# Join distance and travel time back to the main GeoDataFrame
merged_gdf = merged_gdf.merge(
    centroids_gdf[["GEOID", "distance_to_office_miles", "travel_time_to_office_min"]],
    on="GEOID",
    how="left"
)

# Preview the new commute-related fields
merged_gdf[["GEOID", "distance_to_office_miles", "travel_time_to_office_min"]].head()
```

*Partial Output:*

```
   centroid_lat  centroid_lon
0     35.903341    -78.972819
1     35.883118    -78.958137
2     36.005911    -78.811096
3     35.962512    -78.828554
4     35.925008    -78.941546
```

| | GEOID | distance_to_office_miles | travel_time_to_office_min |
|---|---|---|---|
| 0 | 370630020191 | 13.921308 | 23.865100 |
| 1 | 370630020201 | 12.606907 | 21.611840 |
| 2 | 370630018061 | 13.567275 | 23.258186 |
| 3 | 370630018082 | 11.057359 | 18.955473 |
| 4 | 370630020241 | 13.157935 | 22.556460 |

*Sanity Check:*

```
# Validation check
print(merged_gdf[["distance_to_office_miles", "travel_time_to_office_min"]].describe())
✓ 0.0s
```

```
       distance_to_office_miles  travel_time_to_office_min
count                835.000000                 835.000000
mean                  11.168350                  19.145744
std                    5.365078                   9.197277
min                    0.383483                   0.657399
25%                    6.780429                  11.623593
50%                   10.947808                  18.767671
75%                   15.188035                  26.036631
max                   28.967145                  49.657963
```

With commute distance calculated, we can now enrich our dataset with additional neighborhood features such as nearby parks, grocery stores, and bike trails.

Copyright © 2026, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

### *Step 9 | Neighborhood Amenities from OpenStreetMap*

So far, our dataset describes neighborhoods using **Census demographic and housing statistics**.

However, demographic characteristics are only one part of what makes a neighborhood appealing. Families often care about **what amenities are located nearby**, such as parks, grocery stores, and recreational infrastructure.

To capture this information, we turn to **OpenStreetMap (OSM)**.

#### *What Is OpenStreetMap?*

**OpenStreetMap** is a global, crowd-sourced geographic database that contains detailed information about roads, buildings, parks, trails, shops, and many other types of locations.

Much like Wikipedia, OpenStreetMap is maintained by a community of contributors who add and update geographic features around the world.

Because the data are open and freely accessible, analysts can use them to enrich datasets with **local amenities and geographic context**.

In this notebook, we use the Python package **osmnx** to query OpenStreetMap and retrieve features located within our study area.

#### *Amenities That Matter for This Project*

For OperationNextNest, we focus on several amenities that may be particularly important for families:

- **Grocery stores and supermarkets**, including selected large retail stores
- **Parks**, which provide outdoor recreation and green space
- **Bike trails**, which support recreation and alternative transportation

These amenities help describe how **livable and family-friendly** a neighborhood may be.

#### *Creating Amenity Indicators*

To incorporate these features into our dataset, we create simple **binary indicator variables** for each block group:

| VARIABLE | MEANING |
|---|---|
| **HAS_GROCERY_STORE** | At least one grocery or major retail store is present |
| **HAS_PARK** | At least one park is located within the block group |
| **HAS_BIKE_TRAIL** | A bike trail or cycleway is present |

Each variable takes a value of **1 if the amenity is present**, and **0 otherwise**.

These indicators allow us to incorporate neighborhood amenities into later stages of the project, including the **scoring and scenario analysis performed in SAS**.

Copyright © 2026, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

### *Flexible Definitions*

One important characteristic of OpenStreetMap is that amenities are defined using **tags** assigned by contributors.

Because of this, analysts often need to decide which tags should be included when identifying a particular type of location.

For example, a “grocery store” might include several different OSM tags such as:

- supermarket
- convenience store
- department store
- wholesale club

Later in the project, students can experiment with different tag definitions or additional amenities to see how these choices influence the neighborhood rankings.

### *The Code:*

```
# Ensure the study area is in WGS84 (latitude/longitude) for OpenStreetMap queries
merged_gdf = merged_gdf.to_crs(epsg=4326)

# Build a single study-area boundary from all block groups
study_area_boundary = merged_gdf.geometry.union_all()

# Define the OpenStreetMap tags used for each amenity category
amenity_queries = {
    "grocery_store": {
        "amenity": ["supermarket", "convenience"],
        "shop": ["department_store", "wholesale"]
    },
    "park": {
        "leisure": "park"
    },
    "bike_trail": {
        "highway": "cycleway"
    }
}

amenity_data = {}
```

Copyright © 2026, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

```python
for amenity, tags in amenity_queries.items():
    print(f"Fetching {amenity} features from OpenStreetMap...")
    try:
        amenities_gdf = ox.features_from_polygon(study_area_boundary, tags)
        amenities_gdf = amenities_gdf[amenities_gdf.geometry.notna()].copy()

        if amenities_gdf.crs is None:
            amenities_gdf = amenities_gdf.set_crs(epsg=4326)
        else:
            amenities_gdf = amenities_gdf.to_crs(epsg=4326)

        amenity_data[amenity] = amenities_gdf
        print(f"  Retrieved {len(amenities_gdf):,} features for {amenity}.")
    except Exception as e:
        print(f"  Error fetching {amenity}: {e}")
        amenity_data[amenity] = gpd.GeoDataFrame(geometry=[], crs="EPSG:4326")

# Add a temporary block-group ID
merged_gdf = merged_gdf.copy()
merged_gdf["bg_id"] = merged_gdf.index

for amenity, amenities_gdf in amenity_data.items():
    col_name = f"has_{amenity}"

    if not amenities_gdf.empty:
        print(f"Computing intersections for {amenity}...")

        # Add an explicit match flag on the right-hand dataset
        amenities_tmp = amenities_gdf[["geometry"]].copy()
        amenities_tmp["amenity_match"] = 1

        joined = gpd.sjoin(
            merged_gdf[["bg_id", "geometry"]],
            amenities_tmp,
            how="left",
            predicate="intersects"
        )
```

```python
        matched_ids = joined.loc[joined["amenity_match"].notna(), "bg_id"].unique()
        merged_gdf[col_name] = merged_gdf["bg_id"].isin(matched_ids).astype(int)
    else:
        print(f"No {amenity} features found; setting {col_name} = 0 for all block groups.")
        merged_gdf[col_name] = 0

# Drop temporary ID
merged_gdf = merged_gdf.drop(columns=["bg_id"])

# Preview the new amenity indicators
merged_gdf[["GEOID", "has_grocery_store", "has_park", "has_bike_trail"]].head()
```

Copyright © 2026, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

*Partial Output:*

```
Fetching grocery_store features from OpenStreetMap...
  Retrieved 98 features for grocery_store.
Fetching park features from OpenStreetMap...
  Retrieved 738 features for park.
Fetching bike_trail features from OpenStreetMap...
  Retrieved 5,467 features for bike_trail.
Computing intersections for grocery_store...
Computing intersections for park...
Computing intersections for bike_trail...
```

| | GEOID | has_grocery_store | has_park | has_bike_trail |
|---|---|---|---|---|
| 0 | 370630020191 | 0 | 0 | 0 |
| 1 | 370630020201 | 0 | 0 | 1 |
| 2 | 370630018061 | 0 | 0 | 0 |
| 3 | 370630018082 | 0 | 1 | 0 |
| 4 | 370630020241 | 0 | 0 | 1 |

*Sanity Check:*

```python
# A little santiy check
merged_gdf[
    ["has_grocery_store", "has_park", "has_bike_trail"]
].sum()
```

✓ 0.0s

```
has_grocery_store     51
has_park             411
has_bike_trail       385
dtype: int64
```

With neighborhood amenities added to the dataset, we now have one final step: exporting the fully engineered dataset so it can be used for modeling in SAS.

### *Step 10 | Export for Analysis in SAS*

The next notebook in OperationNextNest moves from **data engineering** to **decision modeling**.

In Part 2, we will use **SAS** to evaluate neighborhoods, apply decision constraints, and compare alternative scenarios using the dataset created in this notebook.

To make that possible, we now need to **transfer our dataset from Python to SAS**.

#### *Exporting the Dataset*

The simplest way to share data across programming environments is to export the dataset as a **CSV file**. CSV (Comma-Separated Values) files are widely supported and work well across Python, SAS, and R.

Exporting the dataset as a CSV provides several advantages:

- The file structure is **transparent and easy to inspect**
- It can be imported directly into SAS using LIBNAME statements or PROC IMPORT

Copyright © 2026, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

- It creates a **clean handoff point** between the data engineering and modeling stages of the project

### *What the Dataset Now Contains*

At this point, our dataset contains a rich set of features describing neighborhoods across Wake and Durham counties, including:

- **Census demographic and housing characteristics**
- **Family-related indicators**
- **Estimated commute distance and travel time** to the SAS campus
- **Neighborhood amenity flags** derived from OpenStreetMap
- **Geographic identifiers** that allow the data to be mapped and joined later

Each row represents a **single Census block group**, and each column represents a potential factor that may influence where Lincoln and his family choose to live.

### *The Polyglot Analytics Workflow*

This step marks the handoff point in our **polyglot analytics pipeline**. And here is a reminder of our project workflow:

| Tool | Role | Purpose |
|---|---|---|
| Python | Data Engineering | Build the neighborhood dataset |
| SAS | Scenario Comparison | Evaluate neighborhoods under different priorities |
| R | Interactive Mapping | Visualize geographic patterns and robust candidates |

This approach highlights an important principle in modern analytics: *use the **right tool for the job***.

### *Preparing for Part 2*

The code in this step exports the final dataset to a CSV file that will be used in the next notebook. Once the file is saved, the data engineering stage of OperationNextNest is complete.

Copyright © 2026, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

*The Code:*

```
# ------------------------------------------------------------
# Step 10 — Export dataset for use in SAS
# ------------------------------------------------------------

import re
import os

# Use the workspace path so the notebook works for all learners
workspace_path = os.environ.get("WORKSPACE_PATH", ".")

output_folder = os.path.join(
    workspace_path,
    "OperationNextNest",
    "Data"
)

os.makedirs(output_folder, exist_ok=True)

df_export = merged_gdf.copy()

# ------------------------------------------------------------
# Remove geometry-related columns (not needed for SAS modeling)
# ------------------------------------------------------------

cols_to_drop = [
    c for c in ["geometry", "centroid", "centroid_lat", "centroid_lon", "centroid_lat_lon"]
    if c in df_export.columns
]

if cols_to_drop:
    print("Dropping non-tabular columns:", cols_to_drop)
    df_export = df_export.drop(columns=cols_to_drop)
```

```
# ------------------------------------------------------------
# Convert column names to SAS-friendly format
# ------------------------------------------------------------

renamed = {}

for col in df_export.columns:

    # Replace invalid characters
    new = re.sub(r"[^0-9A-Za-z_]", "_", col)

    # SAS variable names must start with letter or underscore
    if not (new[0].isalpha() or new[0] == "_"):
        new = "v_" + new

    # Limit to SAS 32-character maximum
    new = new[:32]

    renamed[col] = new

df_export = df_export.rename(columns=renamed)

# ------------------------------------------------------------
# Export CSV
# ------------------------------------------------------------

csv_path = os.path.join(output_folder, "operation_next_nest.csv")

df_export.to_csv(csv_path, index=False)

print(f"✅ CSV export complete: {csv_path}")
print("Rows, columns:", df_export.shape)
```

Copyright © 2026, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

*Output:*

```
Dropping non-tabular columns: ['geometry']
✅ CSV export complete: ./OperationNextNest/Data/operation_next_nest.csv
Rows, columns: (835, 43)
```

### *Part 1 Complete | Data Engineering in Python*

At this point, we have successfully constructed an **analysis-ready dataset** describing neighborhoods across the Research Triangle region. Well done!

Using Python, we assembled and integrated multiple public data sources — including Census demographics, housing characteristics, commute estimates, and neighborhood amenities — into a single dataset describing candidate neighborhoods across Wake and Durham counties.

This dataset now provides the foundation for the next stage of **Operation Next Nest**.

But building the dataset was only the first step.

### *Next Mission | Part 2 – Neighborhood Scoring in SAS*

Now we return to the question that launched this project:

**Which neighborhoods best match Lincoln's priorities for his next home?**

In this next phase of the mission, we shift from **data engineering** to **decision comparison**. And… rather than producing a single "perfect" answer, this stage of the analysis helps identify **strong candidate neighborhoods under different preference scenarios**.

This approach allows us to see how recommendations change when what matters most changes. Thus, the mission continues in **Part 2 — PROC OPTMODEL in SAS**.

Let's see which neighborhoods earn a spot on Lincoln's shortlist!

Copyright © 2026, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

# Part 2 | Neighborhood Scoring in SAS

## Mission Background

In Part 1 of **Operation Next Nest**, we focused on data engineering. Using Python, we assembled a dataset describing neighborhoods across Wake and Durham counties by combining multiple public data sources.

This dataset included:

- commute estimates to SAS headquarters
- housing characteristics
- neighborhood economic indicators
- family-oriented demographic measures
- nearby amenities such as parks, grocery stores, and bike trails

By the end of that phase, we had successfully transformed raw geographic and demographic data into a **structured dataset of candidate neighborhoods**.

But building the dataset was only the first step. Now we return to the question that started this project:

**Which neighborhoods best match Lincoln's priorities for his next home?**

Some neighborhoods offer shorter commutes. Others may provide more affordable housing. Some areas may be particularly attractive for families or offer stronger access to parks and amenities.

Choosing among these possibilities requires more than simply browsing the data. It requires a structured way to **compare neighborhoods across multiple priorities**.

In this phase of the mission, we move from **data preparation** to **decision comparison**.

## Primary tool: SAS

The primary tool used in this phase is **SAS**, specifically the PROC OPTMODEL procedure.

PROC OPTMODEL is designed for expressing and solving structured decision problems. It allows analysts to define decision variables, scoring rules, and practical limits in a transparent mathematical framework.

In this project, however, the goal is not to solve a purely mathematical optimization problem. Instead, we use PROC OPTMODEL as a flexible way to implement a **scenario-based neighborhood comparison framework**. In other words – and in this project – PROC OPTMODEL serves as a **structured way to compare neighborhoods under different assumptions** — helping us evaluate tradeoffs rather than declare a single "best" answer.

Using SAS, we will:

- construct a weighted scoring system for neighborhoods
- apply practical limits such as commute time and housing value

Copyright © 2026, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

- evaluate many neighborhoods simultaneously
- explore how different priorities influence which neighborhoods rise to the top

This allows us to move beyond simple filtering and instead explore how **tradeoffs between competing priorities** shape the final recommendations.

## Objective

The objective of this phase is to identify **high-scoring candidate neighborhoods** based on a combination of household priorities.

To accomplish this, we will:

1. Prepare a set of comparison variables describing each neighborhood.
2. Convert key measures to a comparable scale so that different attributes can be combined fairly.
3. Construct a scoring framework that reflects household priorities such as commute time, affordability, family demographics, and access to amenities.
4. Use SAS to evaluate neighborhoods under several different **priority scenarios**.

Rather than producing a single "perfect" answer, the goal is to highlight **strong candidate neighborhoods under different preference structures**.

This approach reflects a common reality in analytics: *changing priorities often leads to different—but still reasonable—solutions*.

## Notebook

*02 – OperationNextNest – PROC OPTMODEL in SAS.sasnb*

This notebook performs all the neighborhood scoring you'll need in SAS.

## What You'll Produce

By completing this phase of the project, you will produce:

- a scoring framework for evaluating neighborhoods
- scenario-based comparisons of candidate neighborhoods
- ranked neighborhood recommendations for several different priority structures
- exported scenario results that will be used for visualization in Part 3

These results represent a shortlist of promising neighborhoods that can now be explored geographically in **Part 3 – Neighborhood Maps in R.**

## Mission 2: Starting Now

The data is ready. The mission is clear. Now it is time to open the SAS notebook and begin comparing neighborhoods for Lincoln's next home. Let's go!

Copyright © 2026, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

### *Step 1 | Load the Dataset*

We begin by loading the dataset created in Part 1. Because all work is performed within the same SAS Viya Workbench environment, SAS can immediately access the file produced in Python. This creates a seamless transition from data engineering to neighborhood comparison.

*The Code:*

```
/*-------------------------------------------------------------*
 | Step 1 | Load the Operation Next Nest data from Part 1
 |
 | Note:
 | The dataset created in Python is stored in the shared
 | workspace so SAS can read it directly. If your folder
 | structure differs, update the path below as needed.
 *-------------------------------------------------------------*/

/* Point SAS to the shared project folder */
%let proj_root = &WORKSPACE_PATH./OperationNextNest;
%let data_path = &proj_root./Data;

/* Import the CSV created in Part 1 */
proc import datafile="&data_path./operation_next_nest.csv"
    out=work.operation_next_nest_raw
    dbms=csv
    replace;
    guessingrows=max;
run;

/* Take a quick look at the imported data */
title "Structure of the imported Operation Next Nest dataset";
proc contents data=work.operation_next_nest_raw order=varnum;
run;

title "First 5 observations from the imported dataset";
proc print data=work.operation_next_nest_raw(obs=5);
run;

title;
```

*Partial Output:*

**First 5 observations from the imported dataset**

| Obs | STATEFP | COUNTYFP | TRACTCE | BLKGRPCE | GEOID | GEOIDFQ | NAMELSAD | MTFCC | FUNCSTAT | ALAND | AWATER | INTPTLAT | INTPTLON | Male_under_5 | Female |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 1 | 37 | 63 | 2019 | 1 | 370630020191 | 1500000US370630020191 | Block Group 1 | G5030 | S | 4885136 | 51127 | 35.9033751 | -78.9763406 | 0 | |
| 2 | 37 | 63 | 2020 | 1 | 370630020201 | 1500000US370630020201 | Block Group 1 | G5030 | S | 7053910 | 190334 | 35.8836714 | -78.959859 | 139 | |
| 3 | 37 | 63 | 1806 | 1 | 370630018061 | 1500000US370630018061 | Block Group 1 | G5030 | S | 3377611 | 93260 | 36.0054078 | -78.809788 | 0 | |
| 4 | 37 | 63 | 1808 | 2 | 370630018082 | 1500000US370630018082 | Block Group 2 | G5030 | S | 4429008 | 16941 | 35.9626638 | -78.8290804 | 130 | |
| 5 | 37 | 63 | 2024 | 1 | 370630020241 | 1500000US370630020241 | Block Group 1 | G5030 | S | 586893 | 0 | 35.9250081 | -78.941546 | 75 | |

### *Step 2 | Prepare Comparison Variables*

After loading the dataset created in Part 1, the next step is to prepare a set of variables that can be used effectively in our neighborhood comparison framework.

The dataset imported from Python already contains many useful attributes describing each block group, including commute estimates, housing values, demographic characteristics, and

Copyright © 2026, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

nearby amenities. However, some of the original column names are quite long, and several variables require light preparation before they can be used in later analysis steps.

To make the data easier to work with, we create a smaller set of **clean, readable variables** that capture the key characteristics we want to compare across neighborhoods.

These variables include:

- **travel_time_min** – the estimated commute time from each block group to SAS headquarters
- **distance_miles** – the straight-line distance to the office (a useful secondary measure of proximity)
- **median_value** – the median housing value for the block group
- **median_income** – the median household income
- **pct_kids_under6** – the percentage of households with young children

These variables provide a concise summary of several important neighborhood characteristics: proximity to work, housing affordability, local economic conditions, and family-oriented demographics.

In addition to these demographic and economic indicators, we also include several **amenity indicators** derived during the Python data engineering stage:

- **has_grocery** – whether a grocery store is located nearby
- **has_park** – whether a park is nearby
- **has_bike** – whether a bike trail is nearby

While each of these variables is informative individually, it is often helpful to summarize them into a simple measure of neighborhood convenience. To accomplish this, we create a new variable called **amenity_count**, which represents the total number of these amenities available near the neighborhood. A higher value suggests that residents have easier access to everyday destinations and recreational opportunities.

Finally, we add a simple numeric identifier called **bg_id** for each block group. This identifier allows us to index neighborhoods efficiently when we later construct the decision comparison model using PROC OPTMODEL.

### *Handling Placeholder Values*

During this step, we also perform a small amount of data cleanup. Some public datasets include placeholder values used to indicate missing or unavailable data. These values may appear as large negative numbers or other unrealistic values.

Because these placeholders do not represent meaningful neighborhood characteristics, we remove or convert them before continuing with the analysis. This ensures that later calculations—such as neighborhood scoring—are based only on valid and interpretable values.

Copyright © 2026, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

### *Preparing for the Comparison Framework*

By the end of this step, we have transformed the raw imported dataset into a clean set of variables that describe each candidate neighborhood in a consistent and easy-to-use format.

These variables will serve as the foundation for the next phase of the analysis, where we construct a **comparable scoring framework** that allows different neighborhood characteristics to be evaluated together.

*The Code:*

```
data work.operation_next_nest_prep;
    set work.operation_next_nest_raw;

    /*-------------------------------------------------------*
     | Create clean aliases for the main variables used in the
     | neighborhood comparison framework
     *-------------------------------------------------------*/

    /* Commute and distance */
    travel_time_min = Travel_Time_to_Office__minutes_;
    distance_miles  = Distance_to_Office__miles_;

    /* Housing and economic measures */
    median_value    = Median_Housing_Value;
    median_income   = Median_Household_Income;

    /* Family-oriented measure */
    pct_kids_under6 = __Households_with_Children_under;

    /* Amenity flags */
    has_grocery = has_grocery_store;
    has_park    = has_park;
    has_bike    = has_bike_trail;

    /* Convert known placeholder values to missing */
    if median_value  < 0 then median_value  = .;
    if median_income < 0 then median_income = .;

    /* Ensure amenity indicators behave like 0/1 flags */
    has_grocery = (has_grocery = 1);
    has_park    = (has_park = 1);
    has_bike    = (has_bike = 1);
```

Copyright © 2026, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

```
    /* Count how many target amenities are present */
    amenity_count = sum(has_grocery, has_park, has_bike);

    /* Add a simple numeric ID for later indexing */
    bg_id = _n_;
run;

/* Quick summary check */
title "Summary statistics for prepared neighborhood comparison variables";
proc means data=work.operation_next_nest_prep
    n nmiss min p25 median p75 max;
    var travel_time_min distance_miles median_value median_income
        pct_kids_under6 amenity_count;
run;

title "First 5 prepared neighborhoods";
proc print data=work.operation_next_nest_prep(obs=5);
    var bg_id travel_time_min distance_miles median_value median_income
        pct_kids_under6 has_grocery has_park has_bike amenity_count;
run;

title;
```

*Output:*

**Summary statistics for prepared neighborhood comparison variables**

**The MEANS Procedure**

| Variable | N | N Miss | Minimum | 25th Pctl | Median | 75th Pctl | Maximum |
|---|---|---|---|---|---|---|---|
| travel_time_min | 835 | 0 | 0.6573987 | 11.6235158 | 18.7676707 | 26.0549978 | 49.6579631 |
| distance_miles | 835 | 0 | 0.3834826 | 6.7803842 | 10.9478079 | 15.1987487 | 28.9671451 |
| median_value | 744 | 91 | 13400.00 | 222050.00 | 308150.00 | 392950.00 | 1092600.00 |
| median_income | 790 | 45 | 14688.00 | 60237.00 | 84040.00 | 116786.00 | 250001.00 |
| pct_kids_under6 | 825 | 10 | 0 | 18.2539683 | 31.0344828 | 42.5433526 | 94.5736434 |
| amenity_count | 835 | 0 | 0 | 1.0000000 | 1.0000000 | 2.0000000 | 3.0000000 |

**First 5 prepared neighborhoods**

| Obs | bg_id | travel_time_min | distance_miles | median_value | median_income | pct_kids_under6 | has_grocery | has_park | has_bike | amenity_count |
|---|---|---|---|---|---|---|---|---|---|---|
| 1 | 1 | 23.8651 | 13.9213 | 537200 | 63475 | 8.0882 | 0 | 0 | 0 | 0 |
| 2 | 2 | 21.6118 | 12.6069 | 361000 | 162067 | 39.2077 | 0 | 0 | 1 | 1 |
| 3 | 3 | 23.2582 | 13.5673 | 210800 | 94205 | 52.5657 | 0 | 0 | 0 | 0 |
| 4 | 4 | 18.9555 | 11.0574 | 321300 | 114851 | 28.9963 | 0 | 1 | 0 | 1 |
| 5 | 5 | 22.5565 | 13.1579 | 207000 | 104276 | 40.6844 | 0 | 0 | 1 | 1 |

## *Step 3 | Filter Candidates and Build Comparable Scores*

With the key comparison variables prepared, the next step is to refine our set of candidate neighborhoods and create a scoring framework that allows different characteristics to be evaluated together.

This step has two primary goals. First, we apply a few simple filters to remove block groups that do not meet our minimum data requirements. Second, we convert several variables into **comparable scoring components** that can later be combined into an overall neighborhood score.

Copyright © 2026, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

### *Filtering Candidate Neighborhoods*

Before building the scoring framework, we first remove a small number of observations that contain missing or invalid values. This ensures that the comparison model operates only on neighborhoods for which all required information is available.

We also apply a practical upper bound on commute time. In this example, we remove neighborhoods with estimated travel times longer than 45 minutes. This threshold is intentionally generous—it simply removes extreme outliers that would not realistically be considered for a daily commute to SAS headquarters.

These filtering steps help ensure that the neighborhoods we compare represent **reasonable candidates for further analysis**.

### *Why Comparable Scores Are Needed*

Once the candidate neighborhoods are defined, we face a common challenge in analytics: how to combine variables that are measured on very different scales.

Consider the neighborhood characteristics we are evaluating:

- commute time is measured in **minutes**
- housing value is measured in **dollars**
- median income is measured in **dollars**
- family demographics are measured as a **percentage**
- neighborhood amenities are represented as a **count**

If we attempted to combine these raw values directly into a single score, the variables with the largest numerical scales would dominate the calculation. For example, housing values measured in hundreds of thousands of dollars would overwhelm variables measured in minutes or percentages.

To prevent this, we convert several of the variables to a **standardized 0–1 range**.

### *Converting Measures to a 0–1 Scale*

Standardizing variables places them on a comparable scale while preserving their relative ordering.

After transformation:

- **1** represents a more favorable value
- **0** represents a less favorable value

This transformation allows us to compare neighborhoods across multiple dimensions without allowing any single variable to dominate the scoring simply because of its units.

We also adjust the direction of certain variables so that **higher values always represent more desirable outcomes**. For example:

- **Lower commute time** produces a higher commute score
- **Lower housing value** produces a higher affordability score

Copyright © 2026, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

- **Higher median income** produces a higher income score
- **Higher percentages of young children** produce a stronger family-oriented score
- **More nearby amenities** produce a higher amenity score

Importantly, this standardization step does **not** replace the original variables. We continue to retain the original measures—such as travel time and housing value—so that the final neighborhood recommendations remain easy to interpret.

Instead, standardized values provide a practical way to combine different neighborhood characteristics into a single scoring framework.

*Preparing for Scenario-Based Comparison*

By the end of this step, each neighborhood is described by a set of **comparable scoring components**. These components will allow us to construct a weighted scoring system that reflects household priorities.

In the next stage of the mission, we will use SAS to evaluate neighborhoods under several different priority scenarios and identify the highest-scoring candidates under each set of assumptions.

*Partial Code:*

```
/*-------------------------------------------------------------*
 | Step 3a | Filter to neighborhoods with usable data and a
 |          reasonable commute range
 |
 | The dataset created in Python contains many block groups,
 | but some may contain missing or invalid values for the
 | variables we plan to use in our scoring framework.
 |
 | We remove observations where:
 |   • required values are missing
 |   • values appear invalid (e.g., <= 0)
 |   • commute time exceeds our practical upper bound
 |
 | The 45-minute threshold is intentionally loose. The goal is
 | simply to remove extreme outliers before we begin comparing
 | neighborhoods more carefully.
 *-------------------------------------------------------------*/

data work.onn_filtered;
    set work.operation_next_nest_prep;

    /* Required variables must be present */
    if missing(travel_time_min) then delete;
    if missing(median_value) then delete;
    if missing(median_income) then delete;
    if missing(pct_kids_under6) then delete;

    /* Remove invalid values */
    if travel_time_min <= 0 then delete;
    if median_value <= 0 then delete;

    /* Apply a practical upper bound on commute time */
    if travel_time_min > 45 then delete;

run;
```

Copyright © 2026, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

```
/*-----------------------------------------------------------------*
 | Step 3b | Create standardized versions of selected variables
 |
 | Our neighborhood characteristics are measured on very
 | different scales:
 |
 |   travel_time_min  → minutes
 |   median_value     → dollars
 |   median_income    → dollars
 |   pct_kids_under6  → percent
 |   amenity_count    → count
 |
 | If we combined these raw variables directly, the larger
 | numeric scales would dominate the scoring calculation.
 |
 | To avoid this, we rescale each variable to a 0–1 range using
 | PROC STDIZE with METHOD=RANGE.
 |
 | IMPORTANT:
 | We keep the original variables AND create standardized
 | versions. This allows the final results to remain easy
 | to interpret in their original units.
 *-----------------------------------------------------------------*/

proc stdize data=work.onn_filtered
            out=work.onn_stdized
            method=range;

   var travel_time_min
       median_value
       median_income
       pct_kids_under6
       amenity_count;

run;
```

*Partial Output:*

**Preview of neighborhoods with scoring components**

| Obs | STATEFP | COUNTYFP | TRACTCE | BLKGRPCE | GEOID | GEOIDFQ | NAMELSAD | MTFCC | FUNCSTAT | ALAND | AWATER | INTPTLAT | INTPTLON | Male_under_5 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 1 | 37 | 63 | 2019 | 1 | 370630020191 | 1500000US370630020191 | Block Group 1 | G5030 | S | 4885136 | 51127 | 35.9033751 | -78.9763406 | 0 |
| 2 | 37 | 63 | 2020 | 1 | 370630020201 | 1500000US370630020201 | Block Group 1 | G5030 | S | 7053910 | 190334 | 35.8836714 | -78.959859 | 139 |
| 3 | 37 | 63 | 1806 | 1 | 370630018061 | 1500000US370630018061 | Block Group 1 | G5030 | S | 3377611 | 93260 | 36.0054078 | -78.809788 | 0 |
| 4 | 37 | 63 | 1808 | 2 | 370630018082 | 1500000US370630018082 | Block Group 2 | G5030 | S | 4429008 | 16941 | 35.9626638 | -78.8290804 | 130 |
| 5 | 37 | 63 | 2024 | 1 | 370630020241 | 1500000US370630020241 | Block Group 1 | G5030 | S | 586893 | 0 | 35.9250081 | -78.941546 | 75 |

**Summary statistics for scoring variables**

**The MEANS Procedure**

| Variable | N | N Miss | Minimum | 25th Pctl | Median | 75th Pctl | Maximum |
|---|---|---|---|---|---|---|---|
| commute_score | 716 | 0 | 0 | 0.4422018 | 0.5980408 | 0.7516448 | 1.0000000 |
| housing_afford_score | 716 | 0 | 0 | 0.6468681 | 0.7257691 | 0.8061990 | 1.0000000 |
| income_score | 716 | 0 | 0 | 0.2006108 | 0.3024430 | 0.4452315 | 1.0000000 |
| kids_score | 716 | 0 | 0 | 0.2607120 | 0.4092201 | 0.5566535 | 1.0000000 |
| amenity_score | 716 | 0 | 0 | 0.3333333 | 0.6666667 | 0.6666667 | 1.0000000 |
| travel_time_min | 716 | 0 | 0.6573987 | 11.6239968 | 18.4066748 | 25.2880450 | 44.8143096 |
| distance_miles | 716 | 0 | 0.3834826 | 6.7806648 | 10.7372270 | 14.7513596 | 26.1416806 |
| median_value | 716 | 0 | 13400.00 | 222550.00 | 309350.00 | 394500.00 | 1092600.00 |
| median_income | 716 | 0 | 18974.00 | 65320.50 | 88846.50 | 121834.50 | 250001.00 |
| pct_kids_under6 | 716 | 0 | 0 | 20.2357654 | 31.7625707 | 43.2059603 | 77.6173285 |
| amenity_count | 716 | 0 | 0 | 1.0000000 | 2.0000000 | 2.0000000 | 3.0000000 |

## *Step 4 | Run Scenario-Based Neighborhood Comparisons with PROC OPTMODEL*

At this stage of the analysis, we are ready to turn our scoring framework into a reusable **scenario-testing tool**.

Copyright © 2026, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

Up to this point, we have prepared a set of variables that describe each neighborhood and converted key measures into comparable scoring components. The next step is to use those components to identify neighborhoods that best match a given set of household priorities.

To accomplish this, we use **PROC OPTMODEL**, a SAS procedure designed for expressing structured decision problems.

In many analytics projects, PROC OPTMODEL is used to solve complex optimization problems such as supply chain design or workforce scheduling. In this project, however, we use it in a slightly different way: as a transparent framework for comparing neighborhood options under different preference scenarios.

Our goal is not to discover a single universally "correct" neighborhood. Instead, we want to identify **strong candidate neighborhoods given a particular set of priorities and practical limits**. This makes PROC OPTMODEL a convenient tool for implementing a **scenario-based neighborhood comparison framework**.

### *The Core Components of the Model*

The decision model in this notebook contains three key elements: a decision variable, an objective function, and a set of constraints.

#### Decision Variable

The decision variable represents whether a particular neighborhood is selected for the final recommendation list.

For each candidate neighborhood *i*, we define a binary variable:

- **x[i] = 1** if the neighborhood is selected
- **x[i] = 0** if the neighborhood is not selected

This allows the model to evaluate many possible combinations of neighborhoods while enforcing the rules we define.

#### Objective Function

The objective function defines what the model attempts to maximize.

In this case, the objective is to **maximize the total weighted score of the selected neighborhoods**. Each neighborhood's score is computed from the scoring components created earlier in the notebook.

Because we choose a fixed number of neighborhoods (n_choice = 10), maximizing the **total score** is equivalent to maximizing the **average score** among the selected neighborhoods.

In practical terms, the model simply identifies the group of ten neighborhoods with the highest overall scores under the specified priorities.

#### Constraints

While the scoring framework reflects household preferences, the constraints ensure that the recommended neighborhoods remain realistic.

In this notebook, we enforce three simple limits:

- exactly **10 neighborhoods** must be selected

Copyright © 2026, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

- selected neighborhoods must fall within a maximum **commute time**
- selected neighborhoods must fall below a maximum **median housing value**

These constraints prevent the model from recommending neighborhoods that would not realistically be considered for the final shortlist.

Together, the scoring framework and constraints allow the model to balance **preferences and practical limits** when selecting candidate neighborhoods.

### *Creating a Reusable Scenario Framework*

To make the analysis flexible, we wrap the decision logic inside a SAS macro called **RUN_SCENARIO**.

This macro allows us to run multiple neighborhood comparisons simply by adjusting a set of scenario parameters.

Each scenario follows the same general process:

1. Start from the scored neighborhood dataset (onn_scores).
2. Apply a set of weights that reflect the relative importance of each neighborhood characteristic.
3. Use PROC OPTMODEL to identify the highest-scoring neighborhoods that satisfy the scenario constraints.
4. Save the selected neighborhoods to a scenario-specific dataset and CSV file for later analysis.

The weights applied in each scenario represent the relative importance of different household priorities.

These include:

- **w_commute** – the importance of a shorter commute
- **w_kids** – the importance of family-oriented demographics
- **w_housing** – the importance of housing affordability
- **w_income** – the importance of higher-income neighborhoods
- **w_amenities** – the importance of nearby parks, grocery stores, and bike trails

In most cases, these weights are designed to sum to **1.0**, ensuring that the overall score remains easy to interpret.

### *The Role of the Decision-Maker*

One of the most important lessons in this step is that the **weights do not emerge from the model**.

They are chosen by the decision-maker.

In other words, the model does not determine what matters most. Instead, it helps us evaluate neighborhood options **given the priorities we specify**.

When the priorities change, the recommended neighborhoods may change as well.

That is precisely what makes this framework valuable: it allows us to explore how different preferences lead to different—but still reasonable—neighborhood recommendations.

In the next step, we will use this framework to run several scenarios that emphasize different priorities and examine how the shortlist of candidate neighborhoods changes.

Copyright © 2026, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

*Partial Code:*

```
/*-----------------------------------------------------------*
 | Step 4 | Build a reusable scenario-testing macro
 |
 | For each scenario, we:
 |   1. Compute a weighted neighborhood score
 |   2. Use PROC OPTMODEL to select the best set of neighborhoods
 |   3. Join the chosen neighborhoods back to the full dataset
 |   4. Export the scenario results for use in Part 3
 |
 | Note:
 | The weights reflect household priorities. They should usually
 | add to 1.0 so that the weighted score remains easy to interpret.
 *-----------------------------------------------------------*/

%macro run_scenario(
    scen_name=,
    n_choice=,
    max_travel=,
    max_value=,
    w_commute=,
    w_kids=,
    w_housing=,
    w_income=,
    w_amenities=
);
```

```
/*-----------------------------------------------------------*
 | Step 4b | Use PROC OPTMODEL to select neighborhoods
 |
 | Model structure:
 |   - x[i] = 1 if neighborhood i is selected
 |   - maximize total weighted score
 |   - choose exactly n_choice neighborhoods
 |   - enforce travel time and housing value limits
 *-----------------------------------------------------------*/
proc optmodel;

    /* Set of candidate block groups */
    set BG;

    /* Input data for each candidate */
    num score{BG};
    num travel{BG};
    num value{BG};

    /* Read the scenario-specific scoring data into OPTMODEL */
    read data work.onn_scores_&scen_name.
        into BG=[bg_id]
             score  = weighted_score
             travel = travel_time_min
             value  = median_value;

    /* Decision variable: 1 if selected, 0 otherwise */
    var x{BG} binary;

    /* Objective: maximize total weighted score */
    max TotalScore = sum {i in BG} score[i] * x[i];

    /* Select exactly n_choice neighborhoods */
    con ChooseLimit:
        sum {i in BG} x[i] = &n_choice.;
```

Copyright © 2026, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

```
    /* Enforce a maximum commute time for any chosen neighborhood */
    con TravelLimit {i in BG}:
        travel[i] * x[i] <= &max_travel.;

    /* Enforce a maximum housing value for any chosen neighborhood */
    con ValueLimit {i in BG}:
        value[i] * x[i] <= &max_value.;

    /* Solve as a mixed-integer linear program */
    solve with milp;

    /*-----------------------------------------------------*
     | Create a dataset containing only the chosen
     | neighborhoods for this scenario
     *-----------------------------------------------------*/
    create data work.onn_choice_&scen_name. from
        [bg_id] = {i in BG: x[i] > 0.5}
        chosen_flag = x[i]
        optmodel_score = score[i]
        optmodel_travel = travel[i]
        optmodel_value = value[i];
quit;
```

*Partial Output:*

**Operation Next Nest | Scenario Base | Chosen Neighborhoods**

| bg_id | GEOID | weighted_score | travel_time_min | distance_miles | median_value | median_income | pct_kids_under6 | amenity_count | has_grocery | has_park | has_bike |
|---|---|---|---|---|---|---|---|---|---|---|---|
| 522 | 371830535251 | 0.74361 | 8.1737 | 4.76798 | 347700 | 132500 | 66.9516 | 2 | 0 | 1 | 1 |
| 833 | 371830536182 | 0.73404 | 15.5161 | 9.05106 | 382000 | 162782 | 74.7917 | 2 | 0 | 1 | 1 |
| 19 | 371830536204 | 0.73084 | 12.0826 | 7.04816 | 422400 | 139597 | 73.2026 | 2 | 0 | 1 | 1 |
| 465 | 371830534112 | 0.72106 | 13.8755 | 8.09407 | 544900 | 180833 | 74.8239 | 2 | 0 | 1 | 1 |
| 191 | 371830525042 | 0.71882 | 5.7601 | 3.36007 | 296800 | 85000 | 58.4665 | 2 | 0 | 1 | 1 |
| 15 | 371830536122 | 0.71507 | 8.5974 | 5.01516 | 311100 | 130568 | 56.9876 | 2 | 0 | 1 | 1 |
| 767 | 371830534261 | 0.71314 | 14.2751 | 8.32712 | 513600 | 182831 | 70.9966 | 2 | 0 | 1 | 1 |
| 759 | 371830536082 | 0.71293 | 12.0134 | 7.00780 | 385900 | 180523 | 59.9198 | 2 | 0 | 1 | 1 |
| 385 | 371830528021 | 0.71135 | 15.3650 | 8.96291 | 193200 | 70256 | 69.0519 | 2 | 1 | 1 | 0 |
| 249 | 371830536203 | 0.70905 | 10.3869 | 6.05905 | 364800 | 127917 | 61.7128 | 2 | 0 | 1 | 1 |

## *Step 5 | Try Different "What Matters Most?" Scenarios*

One of the most important lessons in this project is that different households often value different aspects of a neighborhood. For some families, minimizing commute time may be the most important factor. Others may prioritize housing affordability, family-oriented neighborhoods, or convenient access to parks and local amenities.

Because these preferences vary, our comparison framework must be flexible enough to reflect different priorities. To explore this idea, we run several **scenario-based neighborhood comparisons**, each emphasizing a different set of household preferences.

### *Scenario Design*

In this notebook, we evaluate four different scenarios:

| Scenario | Main Emphasis |
|---|---|
| **Base** | Balanced priorities |
| **CommuteFirst** | Shorter commute |
| **KidFriendly** | Family-oriented neighborhoods |
| **AmenityHeavy** | Parks, groceries, and bike access |

Copyright © 2026, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

Each scenario uses the **same underlying neighborhood data**, but modifies the **weights** applied to the scoring components.

These weights determine how strongly each factor contributes to the final neighborhood score. For example, a scenario that prioritizes commuting will assign a higher weight to the commute score, while a family-focused scenario will place greater emphasis on the demographic characteristics of the neighborhood.

In each case, the weights are chosen so that they sum to **1.0**, making the resulting score easy to interpret as a weighted combination of neighborhood characteristics.

### *Running the Scenario Framework*

Once the weights are defined, the RUN_SCENARIO macro executes the comparison process for each scenario.

The macro performs four key steps:

1. Apply the scenario-specific weights to compute a **weighted neighborhood score**.
2. Use PROC OPTMODEL to identify the **Top 10 candidate neighborhoods** that maximize the weighted score while satisfying the scenario constraints.
3. Save the chosen neighborhoods into a scenario-specific dataset.
4. Export the results to a CSV file for later analysis.

Because the same comparison framework is used for every scenario, the only thing that changes is the **priority structure** reflected in the weights.

This allows us to examine how different assumptions about what matters most can lead to different—but still reasonable—neighborhood recommendations.

### *Comparing Scenario Results*

By running multiple scenarios, we gain a much richer understanding of the neighborhood landscape.

Some neighborhoods may appear consistently across multiple scenarios, suggesting that they perform well across a variety of priorities. Others may only appear when a particular factor—such as commute time or amenities—is given greater importance.

Rather than producing a single definitive answer, this approach helps us identify **clusters of promising neighborhoods** and understand how different preferences shape the shortlist.

These results provide the perfect input for the final phase of the mission.

Copyright © 2026, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

*Partial Code:*

```
/*-------------------------------------------------------------*
 | Step 5 | Run additional scenario-based neighborhood
 |          comparisons
 |
 | Each scenario uses a different set of weights to represent
 | a different household priority structure.
 |
 | By comparing results across scenarios, we can see how the
 | recommended neighborhoods change when what matters most
 | changes.
 *-------------------------------------------------------------*/

/*-------------------------------------------------------------*
 | Scenario 1 | CommuteFirst
 |
 | Heavier weight on commute time and a tighter maximum
 | travel-time limit.
 *-------------------------------------------------------------*/
%run_scenario(
    scen_name=CommuteFirst,
    n_choice=10,
    max_travel=25,
    max_value=900000,
    w_commute=0.45,
    w_kids=0.20,
    w_housing=0.15,
    w_income=0.10,
    w_amenities=0.10
);
```

*Partial Output:*

**Operation Next Nest | Scenario CommuteFirst | Chosen Neighborhoods**

| bg_id | GEOID | weighted_score | travel_time_min | distance_miles | median_value | median_income | pct_kids_under6 | amenity_count | has_grocery | has_park | has_bike |
|---|---|---|---|---|---|---|---|---|---|---|---|
| 522 | 371830535251 | 0.76526 | 8.1737 | 4.76798 | 347700 | 132500 | 66.9516 | 2 | 0 | 1 | 1 |
| 467 | 371830535211 | 0.75679 | 0.6574 | 0.38348 | 409400 | 98073 | 43.0508 | 2 | 0 | 1 | 1 |
| 191 | 371830525042 | 0.75451 | 5.7601 | 3.36007 | 296800 | 85000 | 58.4665 | 2 | 0 | 1 | 1 |
| 469 | 371830535221 | 0.74927 | 5.1931 | 3.02929 | 428500 | 149565 | 63.3842 | 1 | 0 | 0 | 1 |
| 389 | 371830535171 | 0.74601 | 3.7656 | 2.19661 | 13400 | 55182 | 37.0023 | 2 | 0 | 1 | 1 |
| 13 | 371830525082 | 0.74485 | 4.6689 | 2.72354 | 499500 | 165051 | 47.8908 | 2 | 0 | 1 | 1 |
| 15 | 371830536122 | 0.73952 | 8.5974 | 5.01516 | 311100 | 130568 | 56.9876 | 2 | 0 | 1 | 1 |
| 711 | 371830530111 | 0.73667 | 8.0857 | 4.71663 | 326700 | 107721 | 71.4744 | 1 | 0 | 1 | 0 |
| 19 | 371830536204 | 0.73422 | 12.0826 | 7.04816 | 422400 | 139597 | 73.2026 | 2 | 0 | 1 | 1 |
| 810 | 371830537272 | 0.72860 | 6.2837 | 3.66547 | 267800 | 99423 | 46.4968 | 2 | 0 | 1 | 1 |

## Part 2 Complete | Neighborhood Scoring in SAS

At this stage of **Operation Next Nest**, the decision comparison phase of the mission is complete.

In this part of the project, we moved beyond simply preparing neighborhood data and began using it to explore meaningful choices. Using SAS and PROC OPTMODEL, we constructed a structured framework for comparing neighborhoods across multiple priorities.

Through this process, we:

- built a transparent scoring framework for evaluating neighborhoods
- applied practical limits such as commute time and housing value

Copyright © 2026, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

- explored how different household priorities influence which neighborhoods rise to the top
- generated scenario-based results that can now be visualized and compared

Rather than attempting to identify a single "perfect" neighborhood, the goal of this phase was to highlight **strong candidate neighborhoods under different preference scenarios**.

This reflects an important lesson in analytics: *Good models do not replace decisions — they help clarify tradeoffs.*

### *Key Takeaways*

Several important insights emerged during this phase of the mission.

- First, there is rarely a single "correct" solution to a real-world decision problem. Different households may reasonably prioritize different factors when evaluating where to live.
- Second, changing the priority weights in our scoring framework can lead to different—but still reasonable—sets of recommended neighborhoods.
- Finally, structured comparison frameworks like the one implemented with PROC OPTMODEL help make these tradeoffs transparent and easier to explore.

By evaluating multiple scenarios, we gain a clearer understanding of how neighborhood characteristics interact with household priorities.

### *Where We Are in the Workflow*

At this point in **Operation Next Nest**, we have used three different analytical tools to move the mission forward. Our guide:

| Tool | Role | Purpose |
|---|---|---|
| Python | Data Engineering | Build the neighborhood dataset |
| SAS | Scenario Comparison | Evaluate neighborhoods under different priorities |
| R | Interactive Mapping | Visualize geographic patterns and robust candidates |

Together, these steps form a complete analytics workflow: data engineering, decision comparison, and geographic exploration.

### *Preparing for Geographic Exploration*

Each scenario in Part 2 produces a dataset describing the selected neighborhoods along with their associated scores and characteristics. These results provide the foundation for the final stage of the project.

In **Part 3**, we will use **R** to visualize these neighborhoods geographically. By mapping the neighborhoods selected under each scenario, we can explore patterns that may not be obvious from tables alone.

Copyright © 2026, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

This final step will allow us to see:

- where the recommended neighborhoods are located
- which areas appear across multiple scenarios
- which parts of the region offer the most promising opportunities for Lincoln's next home

With the scenario comparisons complete, we are now ready to bring the data to life on the map.

### *Next Mission | Part 3 – Neighborhood Maps in R*

In the next phase of **Operation Next Nest**, we will load the scenario results produced in SAS and explore them using interactive maps.

By visualizing the results geographically, we can better understand how the scenario recommendations relate to the broader layout of the Research Triangle region.

Let's continue the mission in **Part 3** and see where the map leads us.

Copyright © 2026, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

# Part 3 | Neighborhood Maps in R

## Mission Background

Welcome to **Part 3 of Operation Next Nest** — where our analysis finally comes to life on the map.

In Part 2, we used SAS to compare neighborhoods under different sets of priorities. We built a structured scoring framework, evaluated tradeoffs, and identified strong candidate neighborhoods across multiple scenarios.

At that point, we had:

- Scenario-based neighborhood recommendations
- Weighted scores reflecting different priorities
- A shortlist of candidate locations

But those results existed in **tables**. And while tables tell us **which neighborhoods rank highly**, they do not always reveal:

- where those neighborhoods are located
- how they relate to one another geographically
- whether high-performing areas cluster in certain parts of the region

To answer those questions, we need to move beyond tables and into **geographic visualization**.

### *From Tables to Maps*

This is where Part 3 changes the experience.

Instead of reviewing rows and columns, we begin to **explore the results spatially**.

Using R, we transform our scenario outputs into **interactive maps** that allow us to:

- zoom in on specific neighborhoods
- pan across Wake and Durham counties
- explore how different scenarios distribute across the region
- identify patterns that are difficult to see in tabular form

This shift is important.

Maps do not replace the analysis — they make the results easier to understand.

### *What We Gain from Mapping*

By visualizing our results geographically, we can begin to answer deeper questions:

- Do the top neighborhoods cluster near the SAS campus?
- Are certain areas consistently strong across multiple scenarios?
- Which neighborhoods remain attractive even as priorities change?

This last idea becomes especially important.

Rather than focusing on a single scenario, we can look for **neighborhoods that perform well across many scenarios** — what we refer to as *robust candidates*.

Copyright © 2026, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

## Primary tool: R

The primary tool used in this phase is **R**, with a focus on spatial and interactive visualization.

R provides a powerful ecosystem for working with geographic data, including packages such as:

- **sf** for handling spatial data structures
- **tigris** for accessing Census boundary files
- **leaflet** for building interactive web-based maps

While Python helped us build the dataset and SAS helped us compare neighborhoods, R allows us to do something different:

**Explore the results interactively and visually.**

This is not just about creating static charts.

It is about creating maps that allow you to:

- zoom in on specific neighborhoods
- pan across the region
- click on areas to inspect detailed attributes
- explore patterns dynamically

This level of interactivity turns the analysis into something you can **navigate**, not just observe.

## Objective

The objective of this phase is to translate our scenario-based results into **geographic insight**.

To accomplish this, we will:

1. Load the scenario results produced in SAS
2. Join those results to Census block group boundaries
3. Create interactive maps that display neighborhood scores and selections
4. Compare how different scenarios distribute across the region
5. Identify neighborhoods that consistently appear across multiple scenarios

Rather than producing new scores, this phase focuses on **understanding and interpreting** the results we have already generated.

## Notebook

03 – *OperationNextNest – Neighborhood Mapping in R.ipynb*

This notebook performs the full geographic visualization workflow, transforming scenario results into interactive maps that support exploration and interpretation.

## What you'll produce

By the end of this mission, you will have:

- Interactive maps showing neighborhood scores across Wake and Durham counties
- Visual comparisons of scenario-based neighborhood selections

Copyright © 2026, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

- A geographic understanding of where high-performing neighborhoods are located
- Identification of **robust candidate neighborhoods** that perform well under multiple scenarios

These outputs provide the final layer of insight needed to move from:

**"Which neighborhoods scored well?"** to **"Where should Lincoln actually start looking?"**

## Mission 3: Let's Finish Strong!

The data has been prepared. The neighborhoods have been compared.

Now it's time to bring everything together on the map. In this final phase of Operation Next Nest, we move from: **analysis → understanding → decision insight**

Let's open the notebook and see where the data leads.

### *Step 1 | Environment Setup and Package Loading*

Before we begin working with maps, we first need to prepare the R environment by loading the packages that support data import, spatial analysis, and interactive visualization.

In this project, three packages play a central role:

- sf — for working with spatial data and geographic boundaries
- tigris — for accessing Census block group shapefiles
- leaflet — for building interactive, web-based maps

Together, these packages allow us to move from a traditional dataset to a fully **geographic and interactive view of our results**.

If a package is not already installed in your environment, it will need to be installed before it can be used. In some cases, installation may require additional system dependencies, particularly for spatial libraries such as sf.

Once the packages are successfully loaded, we are ready to begin working with both the **scenario results from SAS** and the **geographic boundary data** needed to map them.

Copyright © 2026, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

*The Code:*

```
# Update rlang first (dependency for dplyr)
# install.packages("rlang")

# Install once if needed:
# install.packages(c("readr", "dplyr", "purrr", "ggplot2", "sf", "tigris", "leaflet"))

library(readr)
library(dplyr)
library(purrr)
library(ggplot2)
library(sf)
library(tigris)
library(leaflet)

# Note:
# readr  -> fast CSV import
# dplyr  -> data manipulation
# purrr  -> functional programming helpers
# sf     -> spatial data objects
# tigris -> Census boundary shapefiles
# leaflet-> interactive web maps

# Ensure tigris returns sf objects
options(tigris_class = "sf")

# Cache downloaded shapefiles locally
options(tigris_use_cache = TRUE)
```

*Partial Output:*

```
Linking to GEOS 3.13.1, GDAL 3.10.3, PROJ 9.6.0; sf_use_s2() is TRUE

To enable caching of data, set `options(tigris_use_cache = TRUE)`
in your R script or .Rprofile.
```

## *Step 2 | Prepare the Data and Scenarios*

With the R environment ready, the next step is to bring the **scenario results from Part 2** into the analysis.

In the previous phase, each neighborhood comparison scenario was exported from SAS as a separate file. These files represent different ways of answering the same question:

*What does the "best" neighborhood look like under different priorities?*

Each scenario reflects a different emphasis — such as commute time, family-oriented characteristics, or access to amenities — but all share the same underlying structure.

For every Census block group, the data includes:

- key neighborhood characteristics
- an overall score based on the scenario priorities
- an indicator showing whether the block group was selected among the **Top 10 candidates**

### *Combining Scenarios into a Single View*

To make the results easier to work with, we combine all scenario outputs into a single dataset.

This step is important.

Copyright © 2026, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

Rather than analyzing each scenario in isolation, bringing them together allows us to:

- compare how different priorities influence neighborhood selection
- track which neighborhoods appear across multiple scenarios
- prepare the data for unified geographic visualization

A key detail in this process is preserving the **GEOID** as a character value. This ensures that the dataset can later be joined correctly with geographic boundary files.

We also attach a **scenario label** to each record so that results can be traced back to their originating scenario.

### *Starting Simple: The Base Scenario*

Before moving into more complex comparisons, we begin with a single scenario — the **Base** scenario.

This scenario reflects a balanced set of priorities and provides a useful starting point for validating the data and building an initial map.

Working with the Base scenario allows us to:

- confirm that the scenario data has been loaded correctly
- verify that neighborhood selections are reasonable
- establish a clear reference point before layering in additional scenarios

Once this foundation is in place, we will expand to more advanced visual comparisons across all scenarios.

### *The Code:*

```
options(tigris_use_cache = TRUE)

workspace_path <- Sys.getenv("WORKSPACE_PATH")
if (workspace_path == "") {
  workspace_path <- "/workspaces/myfolder"
}

data_path <- file.path(workspace_path, "OperationNextNest", "Data")

scenario_names <- c("Base", "CommuteFirst", "KidFriendly", "AmenityHeavy")

cat("Using data path:", data_path, "\n")
```

Copyright © 2026, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

```r
# Helper: read one scenario CSV and tag it
read_scenario_csv <- function(scen) {
  file <- file.path(data_path, paste0("onn_result_", scen, ".csv"))
  if (!file.exists(file)) {
    warning("File not found for scenario: ", scen, " at ", file)
    return(NULL)
  }

  # Make sure GEOID comes in as character, not numeric
  read_csv(
    file,
    show_col_types = FALSE,
    col_types = cols(
      GEOID = col_character(),
      .default = col_guess()
    )
  ) %>%
    mutate(scenario = scen)
}

# Read and stack all scenarios
onn_all <- map_dfr(scenario_names, read_scenario_csv)

dplyr::glimpse(onn_all)
```

```r
# Keep only Base scenario for first, simple map
onn_base <- onn_all %>% filter(scenario == "Base")

# Quick sanity check: chosen vs not chosen in Base
onn_base %>%
  count(chosen_flag) %>%
  mutate(pct = round(100 * n / sum(n), 1))
```

*Partial Output:*

A tibble: 2 × 3

| chosen_flag | n | pct |
|---|---|---|
| <dbl> | <int> | <dbl> |
| 1 | 10 | 1.4 |
| NA | 706 | 98.6 |

### *Step 3 | Bring on the Geography*

Up to this point, our analysis has focused on **which neighborhoods perform well** under different scenarios.

But an important question still remains:

**Where are those neighborhoods actually located?**

Copyright © 2026, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

To answer that, we need to move beyond tabular data and connect our results to the **geographic shapes** of the neighborhoods themselves.

### *From Data to Geography*

In order to create a map, each neighborhood must have a defined location and boundary.

These boundaries come from the **U.S. Census TIGER/Line dataset**, which provides official geographic definitions for Census regions across the United States.

Using R, we retrieve the block group boundaries for:

- Wake County, North Carolina
- Durham County, North Carolina

Each block group is represented as a **polygon**, defining the physical area of that neighborhood.

Just as importantly, each polygon includes a **GEOID** — the same identifier used throughout our analysis.

### *Connecting Results to Location*

The key step in this phase is joining the **scenario results** to the **geographic boundaries**.

This is what transforms our dataset from **a list of neighborhoods and scores** into a map-ready dataset with both **attributes and location**

Once the join is complete, each block group contains:

- its geographic boundary
- its neighborhood characteristics
- its scenario-based score
- its selection status

At this point, every observation in our dataset is no longer just a row — it is a **place**.

### *Why This Step Matters*

This step fundamentally changes how we interact with the data. Before this point, we could compare neighborhoods numerically. After this step, we can explore them **spatially**.

That allows us to:

- identify clusters of high-performing neighborhoods
- understand how proximity to the SAS campus influences results
- see how different scenarios distribute across the region

In short, we move from:

**Which neighborhoods look good?** To **Where do the strongest neighborhoods actually exist?**

Copyright © 2026, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

*The Code:*

```
# ---------------------------------------------------------
# Step 3 | Download block group boundaries and join to the
#          Base scenario results
#
# We use Census TIGER/Line boundaries via the tigris package.
# Setting cb = TRUE requests generalized (simplified) boundaries,
# which draw faster and work well for interactive maps.
# ---------------------------------------------------------

# 1. Download North Carolina block group boundaries
nc_bg <- block_groups(
  state = "NC",
  year = 2021,
  cb = TRUE
) %>%
  mutate(GEOID = as.character(GEOID)) %>%
  st_transform(crs = 4326)   # convert to WGS84 for web mapping

# 2. Keep only Wake County (183) and Durham County (063)
wake_durham_bg <- nc_bg %>%
  filter(COUNTYFP %in% c("183", "063"))

# 3. Join the Base scenario results to the block group geometry
onn_base_sf <- wake_durham_bg %>%
  left_join(onn_base, by = "GEOID")

# 4. Sanity checks: confirm the join worked as expected
cat("Total block groups in Wake + Durham:", nrow(wake_durham_bg), "\n")
cat("Rows in joined spatial dataset:", nrow(onn_base_sf), "\n")
cat("Block groups with non-missing weighted_score:",
    sum(!is.na(onn_base_sf$weighted_score)), "\n")

summary(onn_base_sf$weighted_score)
```

```
# Sanity Check

onn_base_sf %>%
  st_drop_geometry() %>%
  summarize(
    total_block_groups = n(),
    matched_scores = sum(!is.na(weighted_score)),
    unmatched_scores = sum(is.na(weighted_score))
  )
```

*Partial Output:*

A data.frame: 1 × 3

| total_block_groups | matched_scores | unmatched_scores |
|---|---|---|
| <int> | <int> | <int> |
| 835 | 716 | 119 |

Copyright © 2026, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

### *Step 4 | Exploring Neighborhood Scores with an Interactive Map*

With the data now connected to geographic boundaries, we can create our **first interactive map**.

In this step, we use the leaflet package to explore neighborhood results directly within a live, navigable map.

This is a meaningful shift in how we engage with the analysis.

Up to this point, we have been working with tables and scores. Now, we can explore the results in a way that reflects how people actually experience geography.

This map allows us to:

- zoom in and out to examine neighborhoods at different levels of detail
- pan across Wake and Durham counties to compare different areas
- view familiar street-level context, including roads, parks, and town names
- click on neighborhoods to inspect key attributes such as score, commute time, and housing value

Rather than reading the results, we can now **interact with them**.

#### *What We Are Visualizing*

In this first view, each Census block group is shaded according to its **weighted neighborhood score**.

This score reflects a combination of:

- commute time
- housing affordability
- income characteristics
- family-oriented demographics
- neighborhood amenities

Higher values indicate neighborhoods that perform better under the selected scenario.

We begin with the **Base scenario**, which represents a balanced set of priorities and serves as a useful baseline for understanding the overall landscape.

#### *Seeing Patterns Emerge*

As you explore the map, patterns begin to emerge.

Instead of isolated values, we start to see:

- clusters of high-performing neighborhoods
- areas where scores are consistently strong or weak
- how proximity to the SAS campus influences results

Copyright © 2026, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

These patterns are difficult to recognize in a table, but become immediately visible in a geographic view.

*Partial Code:*

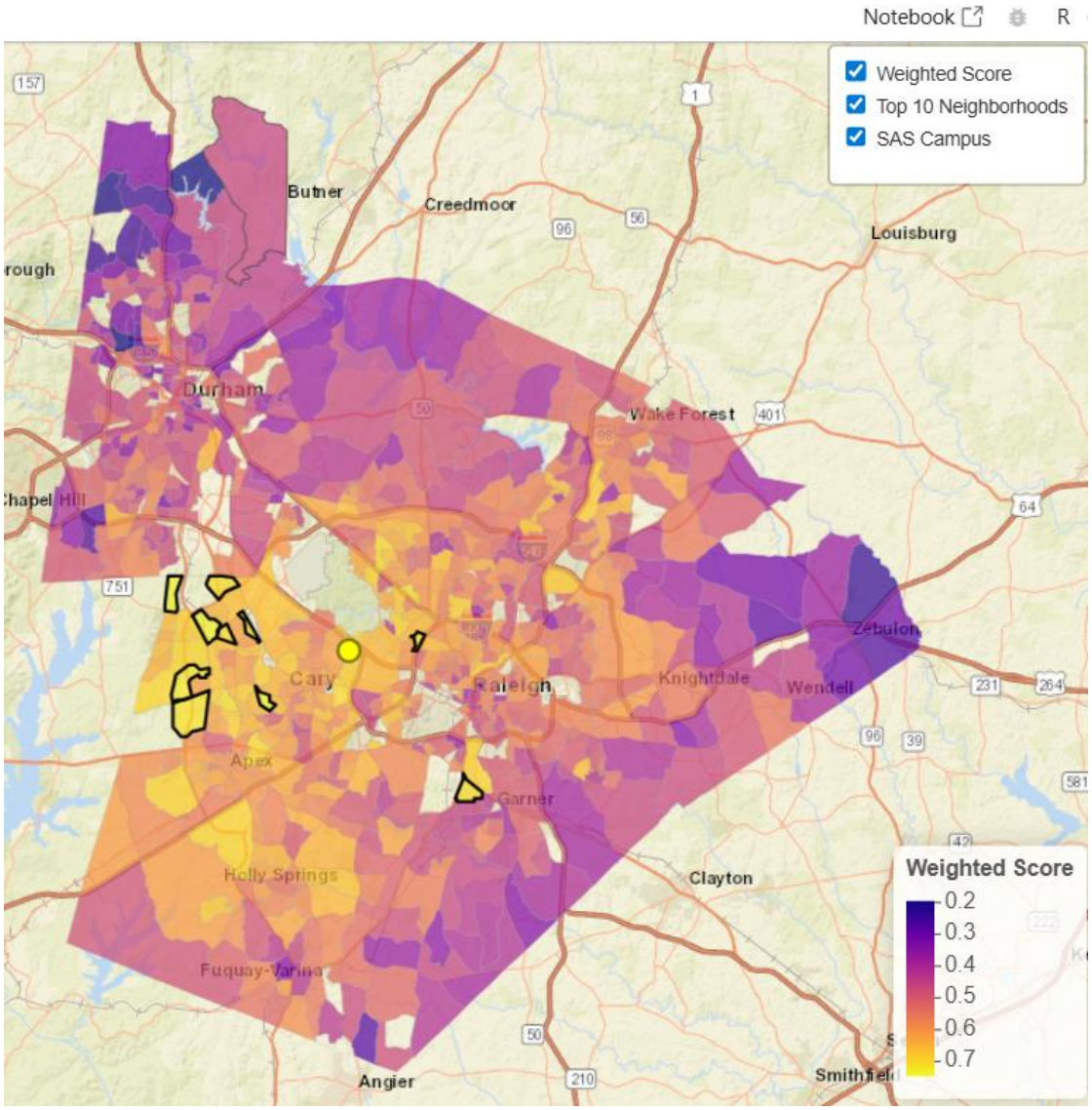
```
# ------------------------------------------------------------
# Step 4 | Interactive map of weighted neighborhood scores
# ------------------------------------------------------------

# Keep geometry in WGS84 for leaflet web maps
onn_base_sf_wake_dur <- st_transform(onn_base_sf, 4326)

# Office location: SAS Campus
office_sf <- st_sf(
  name = "SAS Campus",
  geometry = st_sfc(
    st_point(c(-78.74939336654782, 35.81563890659871)),  # lon, lat
    crs = 4326
  )
)

# Color palette based on non-missing weighted scores
valid_scores <- onn_base_sf_wake_dur$weighted_score[
  !is.na(onn_base_sf_wake_dur$weighted_score)
]

score_pal <- colorNumeric(
  palette = "plasma",
  domain = valid_scores,
  na.color = "transparent"
)
```

…

```
# Legend
addLegend(
  position = "bottomright",
  pal = score_pal,
  values = valid_scores,
  title = "Weighted Score",
  opacity = 1
) %>%

# Optional layer control
addLayersControl(
  overlayGroups = c("Weighted Score", "Top 10 Neighborhoods", "SAS Campus"),
  options = layersControlOptions(collapsed = FALSE)
) %>%

# Initial view
setView(
  lng = -78.85,
  lat = 35.95,
  zoom = 10
)
```

Copyright © 2026, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

*Partial Output:*




*Step 5 | Identifying Consistent Neighborhoods Across Scenarios*

In Part 2, we evaluated neighborhoods under several different priority scenarios. Each scenario produced its own **Top 10 list** of candidate neighborhoods. This leads to an important question: ***which neighborhoods continue to perform well even as priorities change?***

Some neighborhoods may be selected in only one scenario — reflecting a strong fit for a very specific set of preferences. Others may appear repeatedly across multiple scenarios. That repetition tells us something valuable.

*From Scenario Results to Stability*

To capture this idea, we introduce a simple but powerful concept: **stability** — the number of scenarios in which a neighborhood is selected. Rather than focusing on a single ranking, we count how often each block group appears across all scenarios.

Copyright © 2026, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

This allows us to distinguish between:

- neighborhoods selected in only one scenario
- neighborhoods selected in multiple scenarios
- neighborhoods that consistently appear across all scenarios

We then visualize this information using a **stability choropleth**, where shading reflects how often each neighborhood is selected.

### *Why Stability Matters*

Each scenario represents a different definition of what makes a neighborhood desirable:

- shorter commute times
- stronger affordability
- family-oriented characteristics
- access to amenities

A neighborhood that performs well under just one scenario may still be a good fit — but only for that specific set of priorities.

In contrast, neighborhoods that appear across multiple scenarios demonstrate something more:

- they perform well under different assumptions
- they balance competing priorities effectively
- they remain strong candidates even when preferences shift

### *The Key Insight*

This is one of the most useful ideas in the entire project:

- The best candidates are not always the ones that rank highest once.
- They are often the ones that keep showing up.

Copyright © 2026, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

*Partial Code:*

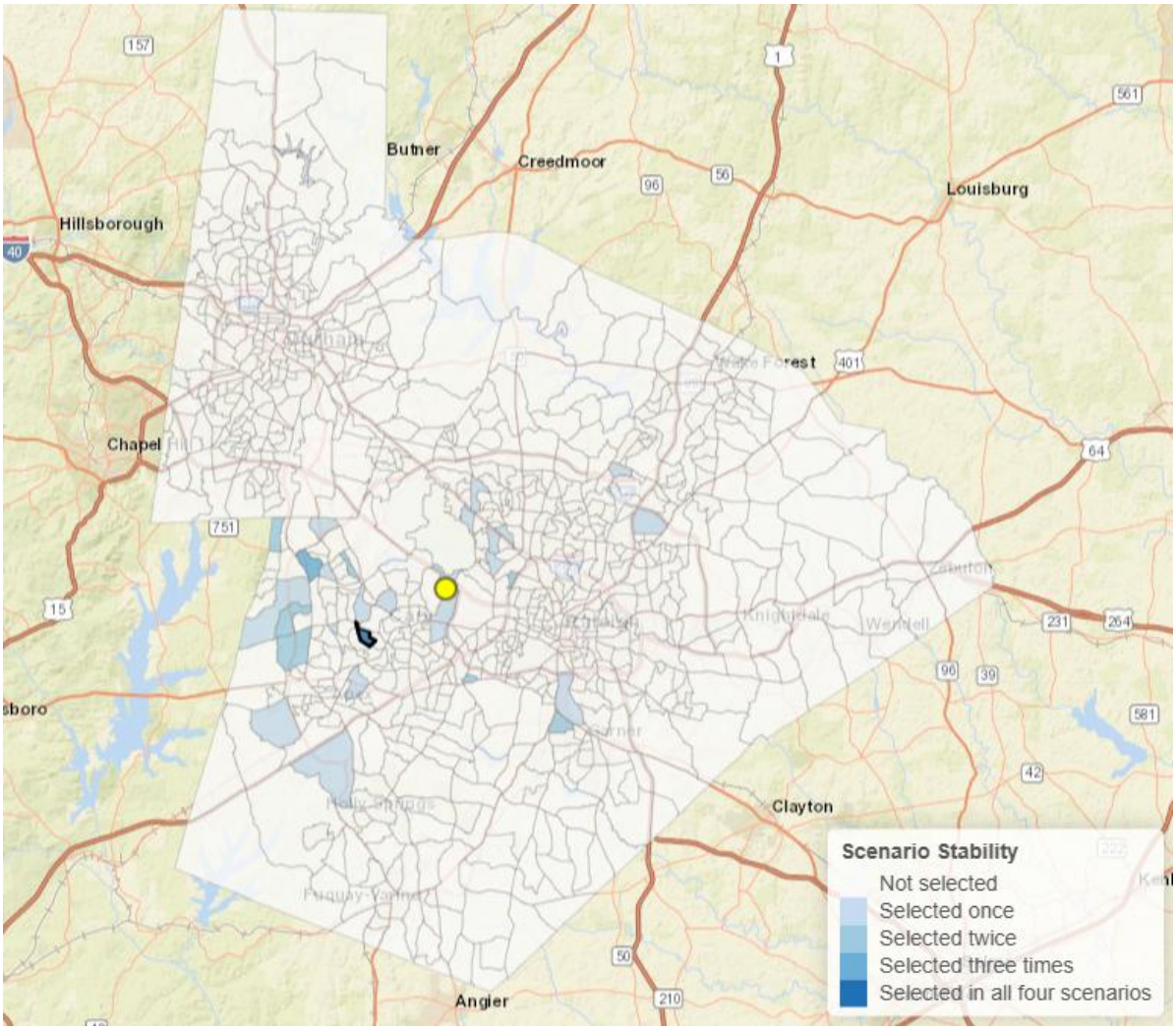
```
# ------------------------------------------------------------
# Step 5 | Stability across scenarios
#
# Goal:
# Identify neighborhoods that remain strong candidates across
# multiple scenario runs by counting how often each block group
# was selected.
# ------------------------------------------------------------

library(dplyr)
library(sf)
library(leaflet)

# 1. Keep only selected neighborhoods from all scenarios
chosen_all <- onn_all %>%
  filter(chosen_flag == 1)

# 2. Count how many scenarios selected each block group
stability_counts <- chosen_all %>%
  count(GEOID, name = "times_selected")

# 3. Join counts back to geometry
stability_sf <- wake_durham_bg %>%
  left_join(stability_counts, by = "GEOID") %>%
  mutate(
    times_selected = ifelse(is.na(times_selected), 0, times_selected)
  )

# 4. Create human-friendly labels in the exact order we want
stability_levels <- c(
  "Not selected",
  "Selected once",
  "Selected twice",
  "Selected three times",
  "Selected in all four scenarios"
)
```

…

```
# Add SAS campus marker
addCircleMarkers(
  data = office_sf,
  radius = 7,
  stroke = TRUE,
  weight = 2,
  color = "black",
  fillColor = "yellow",
  fillOpacity = 1,
  popup = ~name,
  label = ~name
) %>%

# Manual legend with forced order
addLegend(
  position = "bottomright",
  colors = unname(stability_colors[stability_levels]),
  labels = stability_levels,
  title = "Scenario Stability",
  opacity = 1
) %>%

setView(
  lng = -78.85,
  lat = 35.95,
  zoom = 10
)
```

Copyright © 2026, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

*Partial Output:*




*Step 6 | Wrap-Up: What Did We Learn?*

We began this project with a simple but very real question: ***where should Lincoln and his family look for their next home?***

To explore that question, we built a complete analytics workflow — using three tools, each playing a distinct role in the process.

| Tool | Role | Purpose |
|---|---|---|
| Python | Data Engineering | Build the neighborhood dataset |
| SAS | Scenario Comparison | Evaluate neighborhoods under different priorities |
| R | Interactive Mapping | Visualize geographic patterns and robust candidates |

Together, these steps transformed raw data into a **structured decision-support framework**.

Copyright © 2026, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

### *Key Lessons from the Project*

Several important ideas emerged along the way.

**1. Data comes from many places**
Real-world analysis rarely begins with a clean dataset. We combined Census data, geographic boundaries, commute estimates, and amenity information to construct a usable view of neighborhoods.

**2. Geography adds essential context**
Numbers alone can be difficult to interpret. Mapping the results allowed us to see how high-performing neighborhoods cluster and how location influences outcomes.

**3. Priorities shape the answer**
There is no single "correct" neighborhood. As priorities shift — commute time, affordability, amenities, or family-oriented characteristics — the results shift as well.

**4. Stability reveals strong candidates**
Some neighborhoods appeared only once. Others appeared repeatedly. Those repeated appearances highlight **robust candidates** — neighborhoods that remain attractive under different assumptions.

### *What This Project Demonstrates*

This project is not about finding a mathematically perfect answer. It is about using analytics to support **better decision-making**.

By structuring the problem, comparing alternatives, and visualizing the results, we make tradeoffs more visible and easier to understand. Analytics does not replace human judgment. It helps us make **more informed, more confident decisions**.

### *Mission Accomplished?*

We've come a long way. We started with raw data and a simple question: **where should Lincoln and his family look for their next home?**

We built a dataset. We compared neighborhoods under different priorities. We mapped the results and identified strong, consistent candidates. So… mission accomplished?

**Yes — and not quite.**

### *What We've Built*

At this point, we have a strong **decision-support framework**:

- a structured dataset of neighborhoods
- a flexible scoring and comparison system
- interactive maps that reveal geographic patterns
- a method for identifying robust candidates

That's already powerful.

Copyright © 2026, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

But in real-world analytics, this is rarely the end of the story.

### *Where You Could Go Next*

There are many ways this analysis could be extended or refined.

For example, you might:

- incorporate additional data sources
  - school quality
  - crime rates
  - housing inventory
- improve commute estimates using real routing data
- experiment with different scoring approaches or constraints
- explore alternative geographic regions

Each of these directions introduces new questions — and new opportunities to learn.

If you would like to continue your investigation in a more structured way, Part 4 — *Student Mini-Project Details* — provides a set of guided ideas to help you extend and personalize this analysis.

Copyright © 2026, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

# Final Reflection

We began this project with a simple but meaningful question: ***where should Lincoln and his family look for their next home?***

To explore that question, we built a complete analytics workflow that moved from raw data to structured comparison to geographic insight.

Along the way, we:

- assembled data from multiple sources
- built a framework to compare neighborhoods under different priorities
- explored how results change when assumptions shift
- visualized the outcomes to better understand geographic patterns

What emerged was not a single "correct" answer, but something more useful: **a clearer understanding of the tradeoffs involved in the decision.**

## A Polyglot Approach to Analytics

This project also highlights an important idea about modern analytics workflows.

Rather than relying on a single tool, we worked across multiple programming languages — each chosen for the role it plays best:

- **Python** to build and prepare the dataset
- **SAS** to structure and compare decision scenarios
- **R** to explore and visualize the results interactively

All of this work happened within a **single analytics environment**. This is the key idea:

- You don't have to choose one language.
- You can use the language that fits the task — or simply the one you know best.

By combining these tools within a unified platform, we were able to move seamlessly from:

**data → decision → insight**

without switching systems or breaking the workflow.

## What This Really Demonstrates

This project is not about selecting the perfect neighborhood. It is about learning how to approach complex, real-world decisions using data. By structuring the problem, testing different perspectives, and visualizing the results, we create a process that is:

- more transparent
- more adaptable
- and more aligned with real human decision-making

Copyright © 2026, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

## The Bigger Idea

In practice, decisions like this are rarely driven by a single metric. They reflect a combination of:

- personal priorities
- practical constraints
- and subjective preferences

Analytics does not eliminate that complexity. Instead, it helps us **navigate it more effectively**.

## Closing Thought

There is no model that can fully define what “home” means. But with the right tools, the right data, and a thoughtful approach, we can make that search: **more informed, more transparent, and ultimately more human.**

Great job!

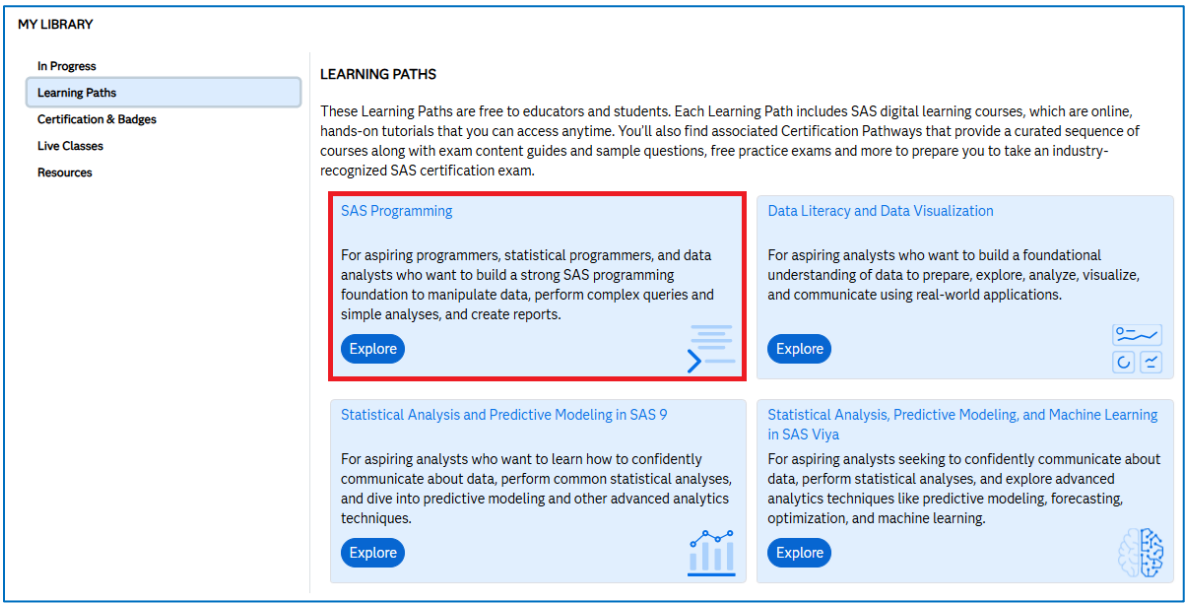
Copyright © 2026, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

# Appendix

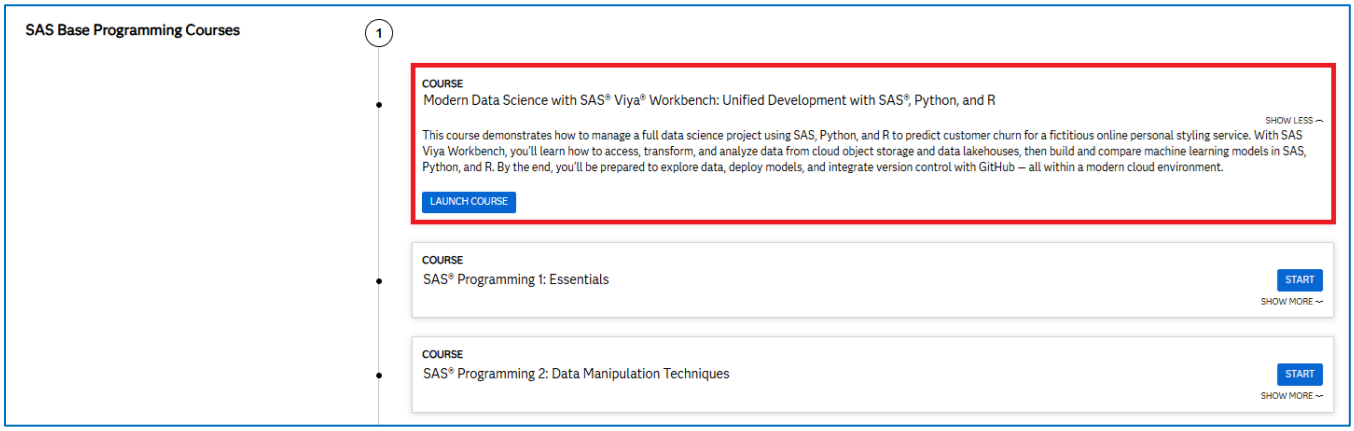
## Appendix A: Next Steps

Did I get you hooked on SAS, R, and Python programming in SAS Viya Workbench? I sure hope so! If yes, your next step could be to take our *Modern Data Science with SAS Viya Workbench, Python, and R* course, available through both the SAS Skill Builder for Students or the SAS Educator Portal.

Once in either learning portal, find the **SAS Programming** learning path and then click **Explore**:




And you can find the course at the top, here:




Go get 'em!

Copyright © 2026, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.